

Databricks – Unity Catalog

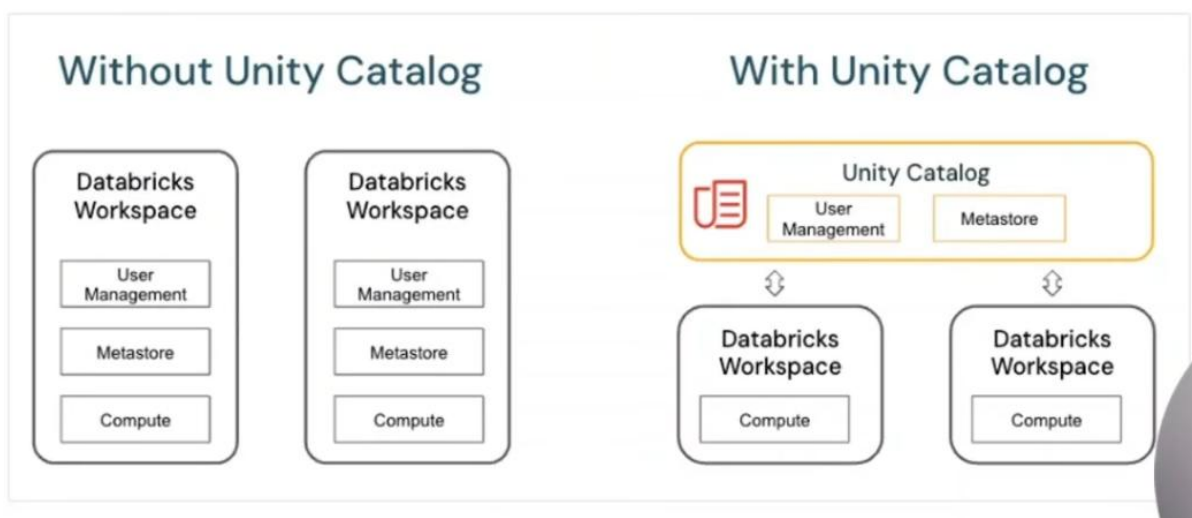
Q1. what is Unity Catalog?

→ It is a centralized data catalog that provides access control, auditing, lineage, quality monitoring, and data discovery capabilities across Databricks workspaces.

KEY POINTS TO HIT

- One metastore per region, shared by many workspaces: **define once, enforce everywhere.**
- Fine-grained control: catalog, schema, table, column (masks) and row (filters) level.
- Automatic **lineage** (down to column level) and **audit logs** with no extra setup.
- Covers structured data, unstructured data (volumes), business metrics and AI models.

Follow-up: How is it different from the legacy Hive metastore? Hive metastore is workspace-local with no central governance, no lineage and table-ACL only. UC is account-level and adds lineage, audit and external-location governance



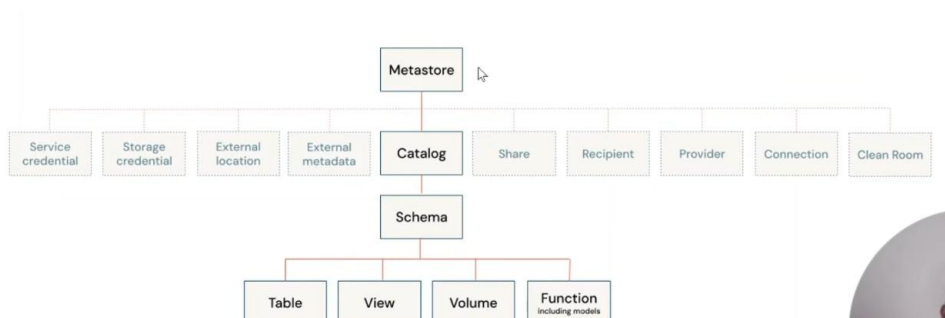
Q2. How are tables and databases arranged in Unity Catalog?

→ Unified catalog for all structured data, unstructured data, business metrics & AI models.

UC uses a hierarchy: **metastore, then catalog, then schema, then objects**. A catalog is usually an environment or domain, like dev, qa and prod. A schema is what we used to call a database, for example raw, bronze or silver. Inside the schema live tables, views, volumes and functions, including registered ML models.

Catalog - dev, QA, Prod

Schema – DB name



3 Level Namespace - <catalog name>.<schema name>.<table/view/model name>

Q3. How is the 3 level namespace work in databricks?

→ Every object is addressed as `catalog.schema.object`, for example `dev.raw_cro.customer`.

`Catalog.schema_name.table_name`

`dev.raw_cro.customer`

Part 2. Compute

Q4. What are different type of clusters that we can create in Databricks? Or what is the purpose of this?

→ 3 types of cluster

All purpose job cluster SQL Warehouse

All this 3 combine call classic compute in databricks.

The newer cluster type is serverless

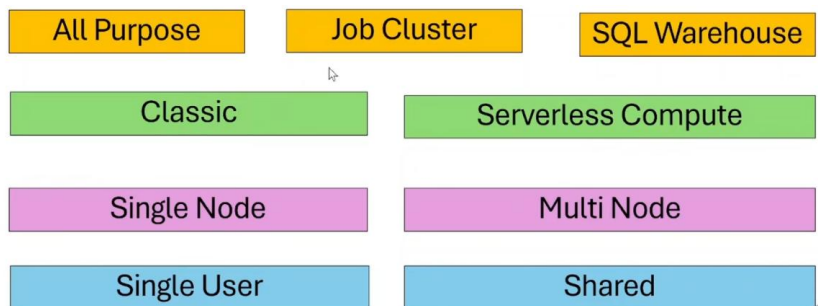
There are three classic compute types:

all-purpose clusters for interactive development,

job clusters for automated production runs, and

SQL warehouses for SQL analytics and BI.

Together these are called classic compute because they run in the customer's cloud account. The newer option is **serverless**, where Databricks runs and manages the compute for you.



Q5. What is the difference between all purpose, Job cluster & sql serverless?

	All-purpose	Job cluster	SQL warehouse
Languages	Spark, SQL, Python, Scala, R, ML	Spark, SQL, Python, Scala, R, ML	SQL only
Purpose	Interactive development and exploration	Automated, production-grade execution	SQL analytics and BI workloads
Typical users	Data engineers, data scientists	Jobs, pipelines, schedulers	BI users, analysts, reporting tools
Lifecycle	Starts on demand, auto-stops when idle	Created per job run, terminated automatically	Starts on demand, auto-stops when idle

	All-purpose	Job cluster	SQL warehouse
Cost	Highest DBU rate, some idle time (slide: about \$0.50/DBU)	Lowest DBU rate, no idle time (slide: about \$0.33/DBU)	Tuned for many concurrent SQL queries

I develop on an all-purpose cluster, but production runs on job compute. A job cluster spins up for the run and terminates right after, so there is no idle cost and the DBU rate is lower. SQL warehouses are for analysts and dashboards, SQL only.

WATCH OUT / LIKELY FOLLOW-UP

The \$0.50 and \$0.33 per DBU figures come from the instructor's slide and are illustrative. Say *"roughly, job compute is noticeably cheaper per DBU"* and avoid quoting exact prices; they vary by cloud, tier and SKU.

		Classic	Serverless
Compute Ownership		Runs in customer's cloud account	Runs in Databricks-managed account
Infra Management		Customer manages networking, quotas, and limits	Fully managed by Databricks
Customization level		Very high (instance type, disk, init scripts)	Limited customization
Operational Overhead		High (Cluster and Nodes configuration, upgrade Runtimes and node types)	Low (Databricks Handles)
Cost		Low DBU and VM are billed separately	High DBUs Only
Data Privacy and Visibility		High, Data stays in our cloud VMs	Low, No visibility where

NO visibility where our data is going

SAY IT LIKE THIS

The difference is where the compute plane lives. In classic, clusters run inside my own cloud subscription, so I control networking, VM types and init scripts, but I also own the operations. In serverless, Databricks runs the compute in its own account, so startup is fast and there is almost no cluster management, but I get less customisation and less direct network visibility. I pick serverless for speed and low overhead, and classic when I need custom libraries, strict network control or cost tuning at scale.

WATCH OUT / LIKELY FOLLOW-UP

Your note said *"no visibility where our data is going"*. Rephrase that: serverless has **lower network-level control**, not that data goes somewhere unknown. Governance still comes from Unity Catalog, and network connectivity and egress controls exist for serverless. Confirm the exact options for your cloud before claiming them.

Architecture sentence to remember: the **control plane** (UI, orchestration, Unity Catalog) lives in Databricks' account; **classic compute plane** is in your cloud account with your workspace storage; **serverless compute plane** is in Databricks' account.

Part 3. Notebooks and developer productivity

6. what magic command and types?

→ It's a specific key set of keyword or command that we mention at the starting of any cell.

%python, %sql, %r – R code, %md – markdown, %sh – shell,

%fs – interact with file systems

%pip – install python packages

7. How can run one Databricks notebook from another databricks notebook?

→ We have 2 options, below command

```
>> %run
```

```
>> dbutils.notebook.run (“./02_notebook_with_param”, 60)
```

	%run	dbutils.notebook.run()
What it does	Includes the other notebook inline in the same context	Starts the other notebook as a separate, ephemeral job run
Variables and functions	Shared: everything defined becomes available	Isolated: nothing is shared
Parameters	Not a clean way to pass them	Pass a dictionary of parameters in; get one string back via <code>dbutils.notebook.exit()</code>
Timeout and errors	None of its own	Timeout in seconds; wrap in try/except for error handling
Monitoring	Part of the parent run	Appears as its own run you can track separately
Use it for	Reusable helper functions and config	Doing real work: parameterised steps with return values

```
# %run must be alone in its cell
```

```
%run ./utils/common_functions
```

```
# separate run: parameters in, one string out, 10-minute timeout
```

```
status = dbutils.notebook.run("./02_load_customer", 600, {"src_name": "crm"})
```

```
# last line of the child notebook
```

```
dbutils.notebook.exit("SUCCESS")
```

If I only want to reuse functions, I use %run, which behaves like an import. If the notebook is a real unit of work that needs parameters, error handling and its own run history, I use `dbutils.notebook.run`. In production, though, I prefer separate tasks in a Databricks job, because the DAG, retries and repair run are managed for me.

8. How shall we select those methods?

→ It depends on behavior

%run – it is to import the function which are reusable.



```
%run "/Workspace/Users/databeli14@gmail.com/PySpark and Databricks Complete Course/03 Databricks/02_Notebook_Run_from_Notebook/02_Notebook_02_Utils_Without_Params"
```

Helper notebook executed Successfully

%run with parameters



```
%run ".02_Notebook_03_Utils_With_Params" $name="Narender" $radius="10"
```

Helper notebook executed with args: Narender radius: 10

dbutils.notebook.run("./02_notebook_with_param", 60) –

Use when are actually doing any operation that notebook. Pass some input para and return some output, do exception handling. All this we can use dbutils.notebook.run() method. It will trigger separate job, we can track it separately.

How to Run One Notebook from Another Notebook ?

Criteria	%run	dbutils.notebook.run
Main Use Case	Import Variables and Functions	Run code separately and get output
Magic Command works in First Line Only	Yes	No
Run Type	Same Thread, No Additional Job	Runs as a Additional Job Run
Input Parameters Supported	Yes	Yes
Output Parameters Supported	No	Yes
Exception Handling Supported	No	Yes

Q. How do you debug code in a Databricks notebook?

SAY IT LIKE THIS

Databricks notebooks have an interactive debugger. I add a breakpoint, run the cell with the debug option, and then step through. **Step in** goes inside a function, **step out** finishes the current function and returns to the caller, and **step over / next line** moves one line at a time. There is also a debug console where I can run small snippets and inspect variables against the paused state.

Q. How do you use AI for development in Databricks?

SAY IT LIKE THIS

I use Databricks' AI coding assistant (Genie Code in your notes). It has two modes. **Chat mode** answers questions and explains code. **Agent mode** takes a requirement, writes the code, runs it, and fixes errors on its own. I still review everything it generates and give it exact, specific prompts, such as table names, expected columns and the output I want.

WATCH OUT / LIKELY FOLLOW-UP

Say you review AI-generated code. Interviewers want to hear judgement, not blind trust. Product names change quickly, so confirm the current name in your workspace.

Secret Scopes

7. How can we save creds in databricks & how can we access them?

→ We create entity named "screte_scope_name".

Where we create screte scope and inside we create multiple key-value pairs. Which will be our screte and passwords that we can access.

For creating screte scope we hv 2 option.

- We can create it backed by databricks, where the creds are store by databricks it self.
- For prod scenarios, we are working with cloud, so there we can be backed by any cloud provider technology as well like **azure key vault**.

Secrets -> Get

Gets the string representation of a secret value for the specified secrets scope and key.

`get(scope: String, key: String): String`



WATCH OUT / LIKELY FOLLOW-UP

Follow-up: *Can someone print the secret?* The REDACTED display is a convenience, not a security boundary. Real protection is scope ACLs and least privilege.

8. How can we connect ADLS store account to databricks ? OR What is the databricks volume?

→ we create volums which maps adls account and to setup this we 1st create credentials in unity catalog and then using that creds we provide permission to adls account then we create external loction in a databricks unity catalog then we finally create a Volume.

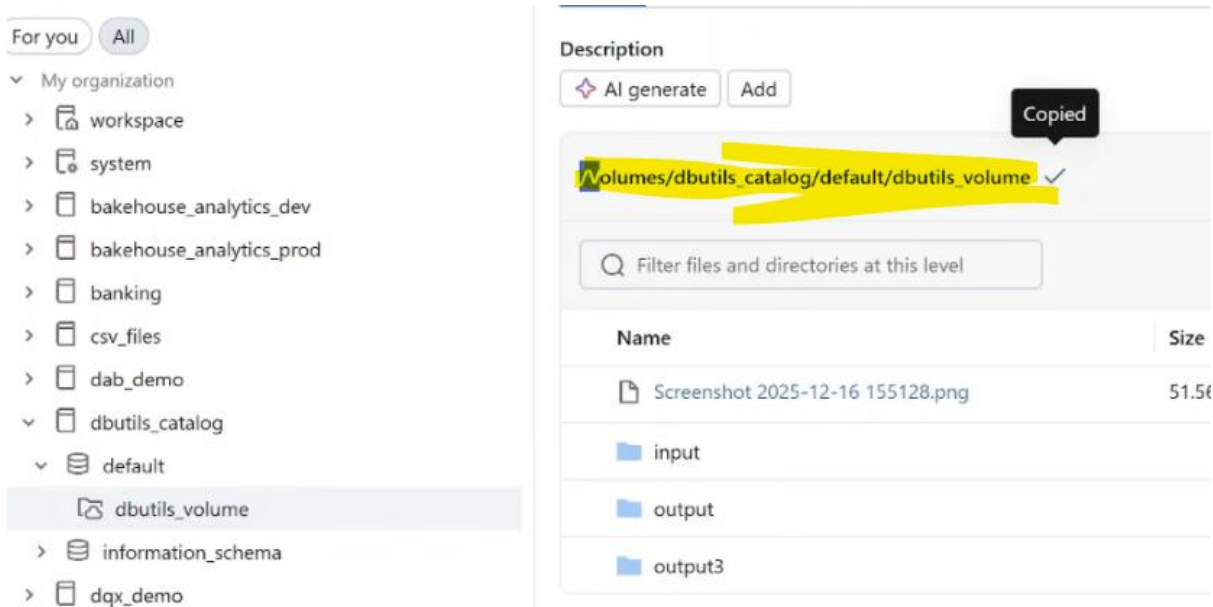
All this steps normally done by admin team.

Once we create volume then that volume maps to container in adls account. We will get path like shown below. With in databricks we can access this container using this path.

Not need to worried about adls path.

SAY IT LIKE THIS

In Unity Catalog the setup goes in four steps, and normally the admin team does it. First, a **storage credential** (backed by an Azure managed identity / access connector). Second, I grant that identity access on the ADLS account. Third, I create an **external location** that ties the credential to a specific ADLS path. Fourth, I create a **volume** on top of it. After that, the volume maps to the container or folder and I just use a path like `/Volumes/catalog/schema/volume/...` I never deal with storage account keys or mount points in code



Q.9 How can be list the file in directory? How to copy some file? Move some file? Copy past complete directory in terms of ADLS Gen2 or S3?

→ SAY IT LIKE THIS

Once the volume exists I use `dbutils.fs.ls` to list, `cp` to copy a file or directory, `mv` to move, `rm` to delete, `mkdirs` to create a directory, and `put` to write a small string to a file.

```
dbutils.fs.ls("/Volumes/prod/raw/landing/input")           # list
dbutils.fs.cp(src, dst, recurse=True)                     # copy file or whole directory
dbutils.fs.mv(src, dst)                                   # move
dbutils.fs.rm(path, recurse=True)                         # delete
dbutils.fs.mkdirs("/Volumes/prod/raw/landing/archive")    # create directory
dbutils.fs.put(path, "hello", overwrite=True)             # write small text file
```

WATCH OUT / LIKELY FOLLOW-UP

Use `recurse=True` to copy or delete a full directory. Same code works for ADLS and S3 because the volume hides the storage path

Part 5. Databricks Jobs (orchestration)

10. How can orchestrate diff notebooks in DB and how we can schedule different entities in databricks like running the NB?

→ we can a job in databricks

SAY IT LIKE THIS

I create a **Databricks job**. A job is a DAG of tasks, and each task can be a notebook, Python script, wheel, SQL, a pipeline, dbt, or another job. I set dependencies between tasks, choose compute per task, and attach a trigger: a cron schedule, file arrival, or continuous. Parameters, retries and notifications are configured at job or task level.

WATCH OUT / LIKELY FOLLOW-UP

Follow-up: *Databricks Jobs vs ADF?* Jobs is native, with the best Spark and Delta integration. ADF is better for metadata-driven, multi-source orchestration across many services

11. How you are pass a value from job to a notebook (NB) ?

→ At job level we can create job parameter manually or during run time also.

This parameter will automatically be populating in the notebook for that job

Then form the notebook side we are receive these value using widgets.

SAY IT LIKE THIS

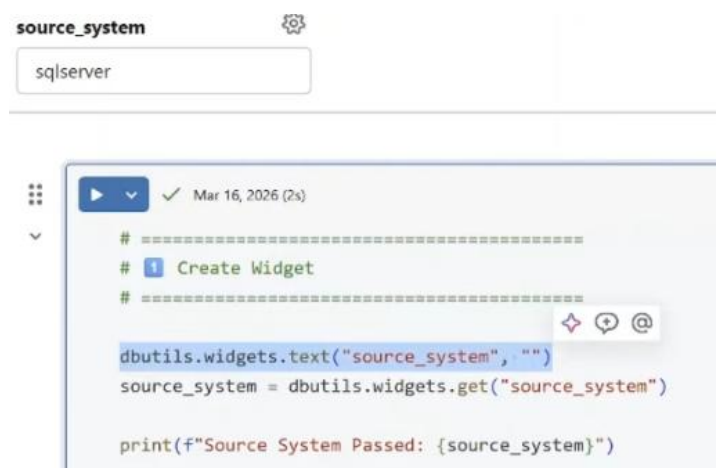
At job level I define **job parameters**, with default values that I can override at run time. Those parameters are passed to the notebook tasks automatically. Inside the notebook I read them with **widgets**: `dbutils.widgets.text` creates the widget with a default value, and `dbutils.widgets.get` reads it.

```
dbutils.widgets.text("src_name", "")          # name, default value

src_name = dbutils.widgets.get("src_name")     # read it in code
```

12. How can you receive any parameter / how can you pass parameterise In notebook?

→ we can do it using widgets. We hv this `dbutils.widgets.text("src_name","")` function. Which takes the name of the widgets and default value also we can provide. It will create widget for ous.then we can receive the value using programtically using `dbutils.widgets.get("src_name")`



13. How can we receive back from notebook to calling job?

→ we have taskValue function

```
dbutils.jobs.taskValues.set(  
    Key = "tables_metadata", value = table_list  
)
```

Value will provide as a output of the job, then the next job will be able to receive it.

In next job we can write input parameter like below

```
{{ tasks.Get_list_of_tables.values.tables_metadata }}
```

SAY IT LIKE THIS

I use **task values**. The producer notebook calls `dbutils.jobs.taskValues.set` with a key and a value. The next task references it as `{{tasks.<task_name>.values.<key>}}` in its input parameters, so no files or temp tables are needed to hand data between tasks.

```
# producer task: Get_list_of_tables  
  
dbutils.jobs.taskValues.set(key="tables_metadata", value=table_list)  
  
# consumer task input / parameter  
  
{{tasks.Get_list_of_tables.values.tables_metadata}}
```

WATCH OUT / LIKELY FOLLOW-UP

Task values must be JSON-serialisable and are size-limited (roughly 48 KB), so pass lists of names or small config, not data. Difference from `dbutils.notebook.exit`: that returns a single string from a notebook run, while task values are key-value pairs for job tasks

NeoBank_Source_to_Silver ☆ Send feedback

Runs Tasks



The screenshot shows the Databricks Jobs interface. At the top, there's a header with 'NeoBank_Source_to_Silver' and a 'Send feedback' link. Below this, there's a 'Runs' tab and a 'Tasks' tab. The 'Tasks' tab is active, showing a task graph. A task named 'Ingest_Data_from_Source_to_Silver' is highlighted, and its configuration is shown in a form below. The task name is 'Ingest_Data_from_Source_to_Silver', the type is 'For each', the input is '{{tasks.Get_List_of_Tables.values.tables_metadata}}', the concurrency is 4, and it depends on 'Get_List_of_Tables'. On the right side, there's a sidebar with 'None', 'Add trigger', 'Job parameter' (with 'run_id' and 'source_system' parameters), 'Compute' (with 'Serverless' and 'Swap' options), and 'Edit parameter'.

Task name* ① Ingest_Data_from_Source_to_Silver

Type* For each

Inputs* ① {{tasks.Get_List_of_Tables.values.tables_metadata}} {}

Concurrency (optional) ① 4

Depends on Get_List_of_Tables X

None
Add trigger

Job parameter
run_id
source_system
Edit parameter

Compute
Serverless
Swap

14. How can we pass parameter in For Each loop?

→ With in for each loop activity, the parameter it can be passed simply as **{{input}}** .

And base on input it will receive the value what ever is provide in outter for each loop as array.

Outter for loop

SAY IT LIKE THIS

A **For each** task takes an input array and runs a nested task once per element. Inside the nested task I reference the current element as **{{input}}**. A pattern I like is a metadata-driven load: the first task returns the list of tables as a task value, then the For each task ingests each table, with a **concurrency** setting (for example 4) so tables load in parallel.

The screenshot shows the Databricks workflow editor. A task named 'Get_List_of_Tables' is connected to a 'For each' loop task named 'Ingest_Data_from_Source_to_Silver'. The 'For each' task is configured with the following settings:

- Task name*: Ingest_Data_from_Source_to_Silver
- Type*: For each
- Inputs*: `{{tasks.Get_List_of_Tables.values.tables_metadata}}`
- Concurrency (optional): 4

Inner For loop

The screenshot shows the 'Job parameters' section for the 'For each' loop task. It displays a table of parameters:

Key	Value
run_id	12345 (pushed down)
source_system	blob (pushed down)
run_id	-(default)
table_metadata	<code>{{input}}</code>

In input, value will receive one by one, which is present in array.

15. What are the defences mechanism to handle failure DB Job? And how we can set retry for diff task in job? How we can rerun fail task in job without rerunning entire job?

→

Task level – we can setup retry mechanism (default 4) (we can set wait duration in hrs).

And we can also run only failed task. (we get **repair run** Butten in job UI)

We can setup email notification on failure. (for every job we hv option called job notification. Inn that we can configure start, success, failure, duration warning)

SAY IT LIKE THIS

There are four layers. First, **task-level retries**, where I set the number of retries and the wait between them for transient problems like a network blip. Second, **Repair run**: if one task fails, I click Repair run in the job UI and it re-runs only the failed task and tasks that depend on it, so I do not repeat expensive successful tasks. Third, **notifications** on start, success, failure and duration warning by email or other channels. Fourth, **timeouts** and *run-if* conditions, so a cleanup or alert task can still run when something upstream fails.

WATCH OUT / LIKELY FOLLOW-UP

Your note says retry default is 4. I could not confirm that, so do not quote a default. Say: *"retries are configurable per task, and I set them based on how transient the failure is."*

Always mention **idempotency**: retries and repair are only safe if the task can run twice without duplicating data (MERGE or overwrite-by-partition, not blind append).

Part 6. Auto Loader

16. What is autoloader?

→ It provides the functionality for to incrementally and efficient process new file without any additional setup.

We get ready to use library. we have to provide the path which we want to monitor. Whenever there will be new file in that path. It will automatically process those new files. Also track the process file so next time it will know whatever the file is process. We don't hv to do any complex setup there.

SAY IT LIKE THIS

Auto Loader is Databricks' built-in way to **incrementally and efficiently ingest new files** from cloud storage. I point it at a path, and whenever new files land it processes only those, remembers what it has already processed, and handles schema inference and evolution. I do not write my own file-tracking logic. It is a Structured Streaming source named `cloudFiles`.

KEY POINTS TO HIT

- Formats: JSON, CSV, Parquet, Avro, ORC, text, and binary files (unstructured data is ingested as binary).
- Processed-file state is kept in the **checkpoint**, giving exactly-once ingestion.
- Works for streaming and for incremental batch (**availableNow**).

17. How does it keep track of new files OR How does he know the new files?

→ there 2 ways

a. Directory listing mode – default

Auto loader find new files by listing input directory which file are there. From checkpoint location it will find which are files are ingested that's how it will keep track. But it is less efficient.

Because src putting more file so it has to store all those file and process it.

b. File notification mode – recommended way

Whenever file put in path, then it generate push notification in event broker then create a queue list of files. Then auto loader will know those are new file. Then only autoloader process only new file.

But to setup this we need some initial setup required from infra Admin.

SAY IT LIKE THIS

By default Auto Loader uses **directory listing**: it lists the input path and checks the checkpoint to see what is already ingested. That is zero setup but gets slower when the source drops very large numbers of files. For high-volume sources I switch to **file notification mode**, where the storage account publishes an event to a queue for every new file and Auto Loader reads from that queue. It needs one-time setup from the infra admin.

Directory listing mode

- Auto Loader identifies new files by listing the input directory.
- Default, No Additional Permissions required
- Less Efficient

File Notification Mode

- File notification mode leverages file notification and queue services in your cloud infrastructure account
- Recommended, Additional setup is required
- Very Efficient



18. What are the diff type of trigger in autoloader.

→

Trigger	Behaviour	Use for	Code
availableNow	Processes all available data, then stops	Incremental batch, scheduled loads	<code>.trigger(availableNow=True)</code>
processingTime	Runs continuously at a fixed interval	Near real-time ingestion	<code>.trigger(processingTime="1 minute")</code>
once	Processes once, then exits	One-time loads (legacy)	<code>.trigger(once=True)</code>

How do you use Auto Loader in code (read and write)?

```
base = "/Volumes/training/my_schema/my_volume/auto_loader/json"

df = (spark.readStream.format("cloudFiles")

    .option("cloudFiles.format", "json")

    .option("cloudFiles.schemaLocation", f"{base}/schema")

    .load(f"{base}/input"))

(df.writeStream

    .option("checkpointLocation", f"{base}/checkpoint")

    .trigger(availableNow=True)

    .toTable("training.auto_loader_demo.demo_table")

    .awaitTermination())
```

19. What is schema location in auto loader?

→ schema location we are providing using `cloudFiles.schemaLocation`. on that location whenever it read new files, so it will automatically keep track of schema of files.

It will know the schema is matching with existing file or not. If mismatch it will raise alarm.

SAY IT LIKE THIS

`cloudFiles.schemaLocation` is a folder where Auto Loader stores the schema it inferred and how that schema has changed over time. On each new file it compares the incoming structure against the stored schema. What happens on a mismatch depends on the schema evolution mode I choose.

WATCH OUT / LIKELY FOLLOW-UP

Your note said a mismatch *"raises an alarm"*. More precisely: in the default `addNewColumns` mode the stream **fails once**, updates the stored schema, and works on restart. In `rescue` mode it never fails; unexpected data goes to `_rescued_data`

20. How do use autoloader in databricks?

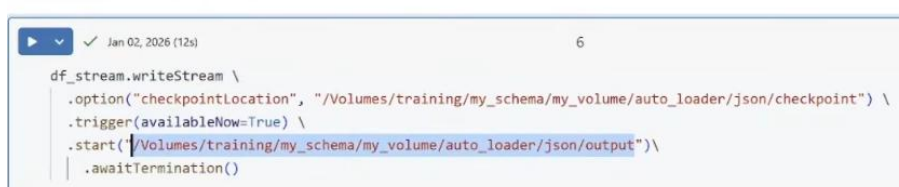
→ read path



```
df_stream=spark.readStream.format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .option("cloudFiles.schemaLocation", "/Volumes/training/my_schema/my_volume/auto_loader/json/schema/")
    .load("/Volumes/training/my_schema/my_volume/auto_loader/json/input/")
```

Write table for autoloader syntax-

Write Stream - Location



```
df_stream.writeStream \
    .option("checkpointLocation", "/Volumes/training/my_schema/my_volume/auto_loader/json/checkpoint") \
    .trigger(availableNow=True) \
    .start("/Volumes/training/my_schema/my_volume/auto_loader/json/output") \
    .awaitTermination()
```

21. What is checkpoint location?

→ It will internally create some files to keep track what are the files we process so far. So that next time based on this checkpoint, it will know that what are new files it needs to process.

Q24. What is the checkpoint location?

SAY IT LIKE THIS

The checkpoint is where Structured Streaming records its progress, including which files Auto Loader has already ingested. If the job stops and restarts, it resumes from the checkpoint and processes only new files, which is how I get exactly-once behaviour.

```
%sql
SELECT * FROM cloud_files_state('/Volumes/training/my_schema/my_volume/auto_loader/json/checkpoint4');
```

Table	+	🔍	🔧	📄
id	path	size	create_time	
1	> dbstorage-prod-33vn7/uc/136589fe-b534-4c44-9a84-913c4945f2d9/6a166556-237b-4ba8-9fc5-9635eeb16e22/_unitystorage/catalog...	794	2026-01-02T12:20	
2	> dbstorage-prod-33vn7/uc/136589fe-b534-4c44-9a84-913c4945f2d9/6a166556-237b-4ba8-9fc5-9635eeb16e22/_unitystorage/catalog...	866	2026-01-02T12:31	
3	> dbstorage-prod-33vn7/uc/136589fe-b534-4c44-9a84-913c4945f2d9/6a166556-237b-4ba8-9fc5-9635eeb16e22/_unitystorage/catalog...	877	2026-01-02T12:31	
4	> dbstorage-prod-33vn7/uc/136589fe-b534-4c44-9a84-913c4945f2d9/6a166556-237b-4ba8-9fc5-9635eeb16e22/_unitystorage/catalog...	874	2026-01-02T12:31	

22. What are options to handle schema in autoloader?

→ option1. We can provide the schema. Which is preferred for prod load.

```
Schema = StructType([
    StructField("id", IntegerType(), Nullable=False),
    StructField("name", StringType(), Nullable=True)
])
```

Option 2. Not provide the schema. So, by default it will give string type for all columns.

SAY IT LIKE THIS

Two options. **Option 1: provide the schema explicitly.** This is what I do in production, because the schema is stable, there is no inference cost, and bad data is caught. **Option 2: let Auto Loader infer it.** For JSON and CSV it reads everything as strings by default unless I set `cloudFiles.inferColumnTypes` to true. That is good for exploration and bronze layers where sources change often.

23. How can we handle corrupted data in autoloader? What is the rescue concept in autoloader?

→ `.option("cloudFiles.schemaEvolutionMode","rescue")`

`_rescued_data` is on by default, but the default **mode** is `addNewColumns` when you do not provide a schema.

Whenever it finds any corrupted data or it finds that the schema is not matching then those particular rows it will put in the rescue data column itself.

SAY IT LIKE THIS

Auto Loader adds a column called `_rescued_data`. Any value that does not fit the schema, such as an extra column, a type mismatch or a column-name case mismatch, is captured there as JSON instead of being lost or crashing the pipeline. If I set the schema evolution mode to `rescue`, the schema never changes and the stream never fails on new fields. In production I use an explicit schema plus rescue mode, and I send rows with a non-null `_rescued_data` to a quarantine table for review.

```
df = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "rescue")
    .option("cloudFiles.schemaLocation", f"{base}/schema")
    .load(f"{base}/input"))

good = df.filter("_rescued_data IS NULL")
bad  = df.filter("_rescued_data IS NOT NULL")  # route to quarantine table
```

Handle Schema Changes - Rescue Data

```
spark.readStream.format("cloudFiles") \
  .option("cloudFiles.format", "json") \
  .option("cloudFiles.inferColumnTypes", "true") \
  .option("cloudFiles.schemaEvolutionMode", "rescue") \
  .option("cloudFiles.schemaLocation", "/Volumes/training/my_schema/my_volume/auto_loader/json/schema5/") \
  .load("/Volumes/training/my_schema/my_volume/auto_loader/json/input/") \
  .writeStream \
  .option("checkpointLocation", "/Volumes/training/my_schema/my_volume/auto_loader/json/checkpoint5") \
  .trigger(availableNow=True) \
  .table("training.auto_loader_demo.demo_table5") \
  .awaitTermination()
```

id	quantity	remarks	_rescued_data
1	5	Repeat business	["order_status":"DELIVERED","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5_scl
2	1		> ["order_status":"CANCELLED","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5
3	3	Customer and price missing	["order_status":"PENDING","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5_sche
4	4	Bulk discount applied	["order_status":"DELIVERED","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5_scl
5	null	Quantity not finalized	["order_status":"ON_HOLD","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5_sch
6	1	International order	> ["order_status":"PROCESSING","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5
7	2	Priority delivery	["order_status":"SHIPPED","file_path":"/Volumes/training/my_schema/my_volume/auto_loader/json/input/sales_5_scl
8	3	Bulk order	null
9	2	Price pending approval	null
10	null	Quantity to be verified	null
11	1	Assembly required	null
12	5	SKU not mapped	null
13	2	Installation scheduled	null
14	1	Educational purchase	null

24. How do you debug any code?

→ Databricks and notebooks, we can add breakpoint and then debug our code.

While running any notebook, we get a debug cell option. Using that can do debug.

Debug options like

- a. Stepin – it takes us inside a function.
- b. stepout - it will complete the execution and then come out of that function.
- c. Go to next line – check step by step the code.

Apart from that we get a debug console where we can run any small code as well.

25. How do you use AI from dev in DB?

→ we can use Genie code. We can give exact prompt what we want accordingly.

There are 2 modes.

1. Chat mode – ask any question
2. Agent mode – based on requirement we provide based on that it will write the code. It will execute the code. Fix the code.

26. If you are working on optimization on any job or task. How do you know how much optimization done?

→ we can calculate exact cost of any job running databricks using sql.

We can take job run id. Based on id, it will fetch the details from 2 tables.

System.billing.usage table – we get info. About start time, end time, how many dbu are used. What is the SKU name (sku name means type of compute).

System.billing.list_prices – which has the rate of those dbu's.

SAY IT LIKE THIS

I calculate the exact cost of a run from system tables. Using the **job run ID**, I read `system.billing.usage`, which gives the start and end time, DBUs consumed and the SKU name (the compute type), and I join it to `system.billing.list_prices` to get the price per DBU for that SKU. Cost is DBUs times price. I run the job before optimisation and note runtime and cost, apply the change, run the same job on comparable data, and compare the two numbers.

```

select
  sum(x.cost) as cost_in_dollar
from (
  select
    u.usage_quantity, p.price, u.usage_quantity * p.price as cost
  from (
    select
      usage_start_time, usage_end_time, usage_quantity, usage_unit, sku_name
    from system.billing.usage
    where usage_metadata.job_run_id = '481719574637711'
  ) u
  join (
    select
      pricing.effective_list.default ✨💬@, sku_name
    from system.billing.list_prices
    where price_end_time is null
  ) p
  on u.sku_name = p.sku_name
) x

```

Table ▾		+
	.00 cost_in_dollar	
1	0.442605	

Before doing the optimization we will run this job and note down the time and cost. And after that we perform what optimization and run this same job(sql query in img) and compare.

27. What is lakeflow connect and when shall we use that?

→ It is the data ingestion service from the databricks.

Whenever we got to “**Job & Pipelines**” we will get 3 options.

- Ingestion pipeline
- Etl pipeline
- Job

Ingestion pipeline – it is a lakeflow connect. Us whenever we ingest the src and we need to ingest the data into our databricks env, then we use lakeflow connect.

If we want manage connector where we don’t want to write any code, we just want to connect with src. After that we just select our table that we want to ingest, and based on whether we want full load or incremental load or CDC.

We configure those options once in lakeflow connect then ingestion will take care automatically.

Whenever we want to reusable metadata-driven framework. In that case lakeflow connect will not use. We can do using ADF.

28. What asset bundle or Declarative automation or where to use?

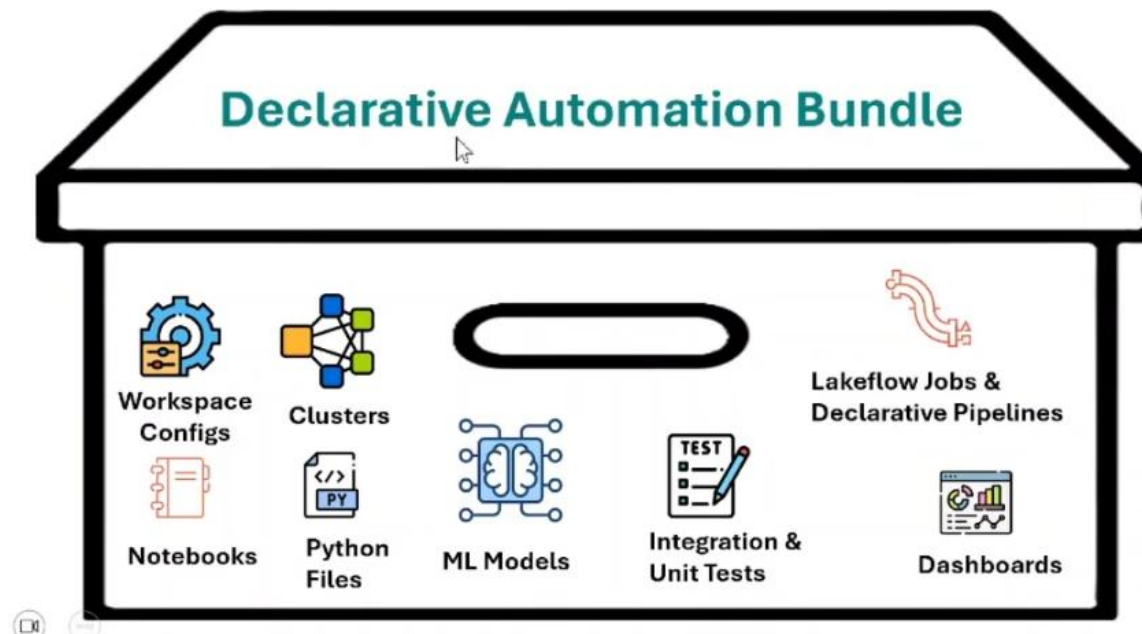
→ It enable **Software Engineering Lifecycle for Data/AI**

They allow us to manage complex Databricks project as code, streamlining.

- Source control / Code review - manage everything via Git.
- CI/CD – Automate deployments across dev,QA, prod env

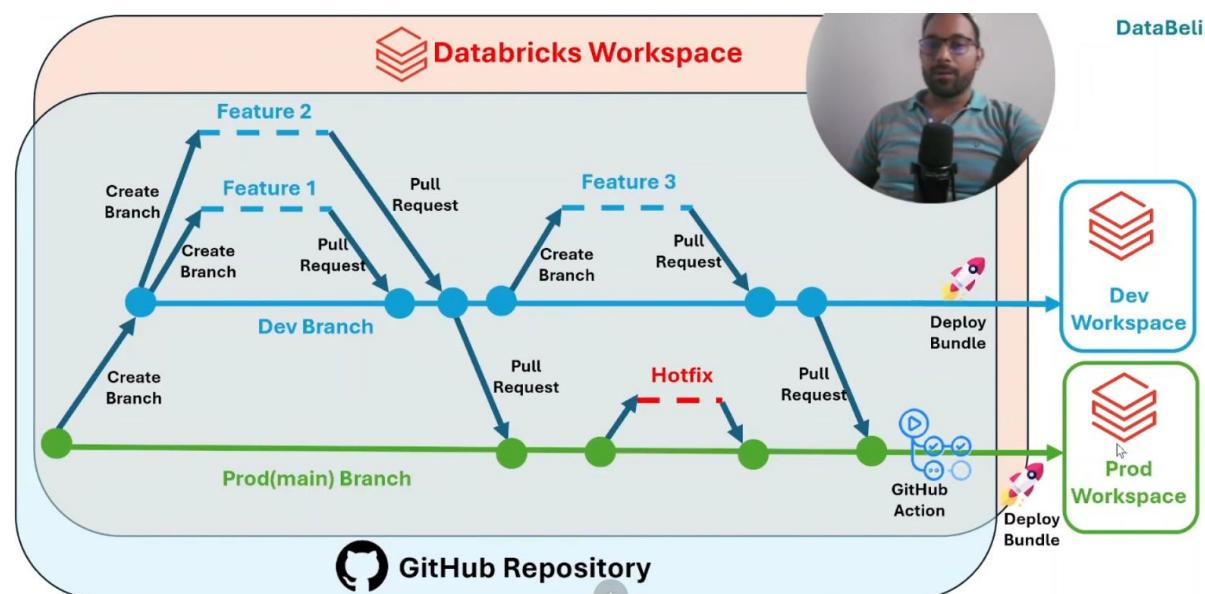
29. What object we can give into a declarative automation bundle?

→



30. How does the CICD process end-to-end?

→



When we are using automation bundles, so we hv connected our repo with our databricks workspaces and there we have main branch which is out prod branch.

Form main branch we created dev and QA branch.

Whenever we are working in new development, enhancement. We create feature branch from QA. Main and QA is in sync. We update feature branch then raise PR to dev. Change are deploy to Dev databricks workspace through Databricks asset bundles.

Once dev complete. Dev to qa raise PR. Then QA to Prod raise PR.

SAY IT LIKE THIS

The repo is connected to our Databricks workspaces. **Main is the production branch**. For any new development or enhancement I create a **feature branch**, make changes, and raise a pull request into the **dev** branch. After review and automated checks, the change is deployed to the **dev workspace** using the bundle. When dev is validated I raise a PR from dev to **QA**, which deploys to the QA workspace for testing, and finally a PR from QA to **main**, where a **GitHub Actions** pipeline deploys the bundle to the **prod workspace**.

Step	Action	Result
1	Create feature branch, develop	Isolated change
2	PR feature to dev (review, unit tests)	Bundle deployed to dev workspace
3	PR dev to QA (integration tests)	Bundle deployed to QA workspace
4	PR QA to main (approval)	GitHub Action deploys bundle to prod workspace

WATCH OUT / LIKELY FOLLOW-UP

Your notes say the feature branch is cut from QA, while the slide shows feature branches off dev, merging back into dev. Pick one flow and describe it consistently. The version above (feature to dev to QA to main) is the safest to explain.

31. What is Hot fix?

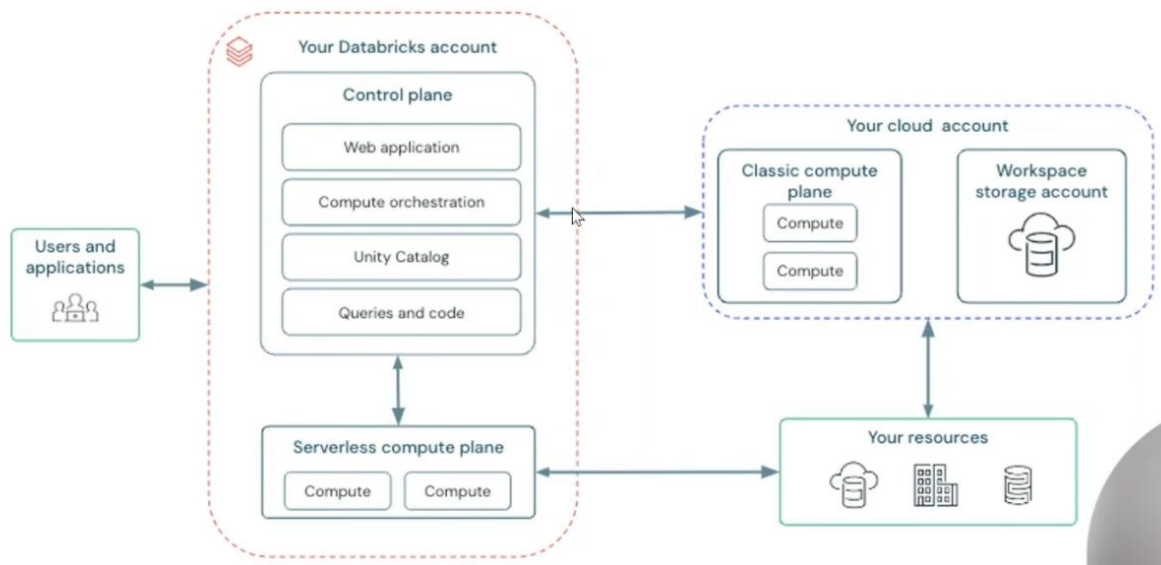
→ Hot fix is some urgent and quick issue that we can fix quickly with going to entire CI/CD process.

Create feature branch from prod. Do changes and push to prod.

SAY IT LIKE THIS

A hotfix is an urgent production fix that cannot wait for the normal dev to QA to prod promotion. I create a **hotfix branch directly from main (prod)**, make the minimal fix, raise a PR into main with review, and the pipeline deploys it to prod. Afterwards I **merge the fix back into dev and QA** so the next release does not overwrite it.

Classic Workspace Architecture



32. What is data quality ready to use feature available in databricks?

SAY IT LIKE THIS

Unity Catalog has **data quality monitoring**. It has two parts. **Anomaly detection** checks **freshness** (how recently a table was updated) and **completeness** (whether the number of rows written in the last 24 hours is what we expect). **Data profiling** gives summary statistics for a table's data, and can also track GenAI apps, ML models and serving endpoints. The health status shows right in Catalog Explorer, so a table can show as unhealthy if it is stale or incomplete.

WATCH OUT / LIKELY FOLLOW-UP

Complement it with rule-based checks: **NOT NULL** and **CHECK** constraints on Delta tables, and expectations in declarative pipelines (drop, fail or warn on violation). Interviewers like hearing both monitoring and enforcement.

Q28. Which system tables should you know?

System table	What it gives you
system.billing.usage	All billable usage: DBUs, SKU, job run ID, time window
system.billing.list_prices	Historical SKU price log (new row whenever a price changes)
system.access.audit	Audit events across workspaces in the region
system.access.table_lineage	A record for each read or write on a UC table or path
system.lakeflow.jobs / job_tasks / job_run_timeline	Jobs, their tasks, and start and end times of runs
system.compute.warehouses / warehouse_events	SQL warehouse configuration history and start, stop, scale events

System table	What it gives you
system.query.history	Every query run on SQL warehouses and serverless compute
system.data_classification.results	Column-level detection of sensitive data classes

Part 10. Rapid-fire round (one-liners)

Question	One-line answer
Unity Catalog in one line?	Central governance for data and AI assets across workspaces.
Namespace format?	<code>catalog.schema.object</code>
Dev vs prod compute?	All-purpose for development, job compute for production.
Which compute is SQL only?	SQL warehouse.
Cheapest for scheduled pipelines?	Job compute: created per run, no idle time.
<code>%run</code> vs <code>notebook.run</code> ?	Inline import vs separate parameterised run.
Prod secret store?	Azure Key Vault-backed secret scope.
How to reach ADLS in UC?	Storage credential, external location, volume.
Notebook param mechanism?	Widgets (<code>dbutils.widgets.text / .get</code>).
Return value between tasks?	<code>dbutils.jobs.taskValues.set / {{tasks.x.values.y}}</code> .
Loop over a list in a job?	For each task, element is <code>{{input}}</code> .
Rerun only failed tasks?	Repair run.
Auto Loader format name?	<code>cloudFiles</code>
Default file detection?	Directory listing; file notification for scale.
Best trigger for incremental batch?	<code>availableNow=True</code>
Bad rows in Auto Loader?	<code>_rescued_data</code> column.
Prove optimisation savings?	<code>system.billing.usage</code> joined with <code>list_prices</code> by job run ID.
Deploy jobs as code?	Declarative Automation Bundles via CLI and GitHub Actions.

Part 11. Gaps to cover next (likely asked, not in your notes)

Your session was strong on platform features. Data engineering interviews usually also test these Spark and Delta topics, so prepare a short answer for each:

Topic	What to be ready to explain
Delta Lake internals	Transaction log, ACID, time travel, MERGE , schema enforcement vs evolution
Performance	OPTIMIZE , liquid clustering vs Z-ORDER and partitioning, VACUUM , small-file problem
Spark tuning	Shuffle, data skew, broadcast joins, AQE, caching, spill
Medallion architecture	Bronze, silver, gold: what goes in each, and where quality checks happen
Declarative pipelines (DLT)	Streaming tables vs materialized views, expectations
Incremental patterns	SCD Type 2 with MERGE , CDC handling, idempotent loads
Structured Streaming	Watermarks, stateful operations, exactly-once semantics
Security	Row filters, column masks, service principals, least privilege

Part 12. Corrections made to your original notes

Original note	Issue	Interview-safe version
<code>dbutils.notebook.run</code> , <code>Dbutils.fs</code>	Typos; the API is case-sensitive	<code>dbutils.notebook.run</code> , <code>dbutils.fs</code>
Q7 and Q8 numbered twice	Numbering restarted	Renumbered Q1 to Q34 in logical order
Retry mechanism default is 4	Not confirmed	"Retries are configurable per task"
Rescue mode is the default	Mixed up two settings	<code>_rescued_data</code> is on by default; default mode is <code>addNewColumns</code>
Mismatch "raises an alarm"	Imprecise	Fails once in <code>addNewColumns</code> ; never fails in <code>rescue</code>
Trigger Once listed as an option	Deprecated	Use <code>availableNow</code>
<code>IntegerType</code> , <code>Nullable=</code>	Typos	<code>IntegerType</code> , <code>nullable=</code>
Directory listing "stores all files"	Imprecise	Listing cost grows with the number of files in the directory
Serverless: "no visibility where data goes"	Overstated	Lower network-level control; governance still via UC

Original note	Issue	Interview-safe version
\$0.5 and \$0.33 per DBU	Slide figures, not official	Quote as "roughly cheaper per DBU"
Feature branch from QA; hotfix "skips CI/CD"	Inconsistent with slide	Feature to dev to QA to main; hotfix skips promotion, not review
Classic workspace architecture image under hotfix	Misplaced	Belongs with classic vs serverless (control plane vs compute plane)
Cost query used only <code>price_end_time is null</code>	Wrong for historical runs	Join on usage time within the price validity window

Azure + DataBricks Interview Questions From Real Project Scenarios

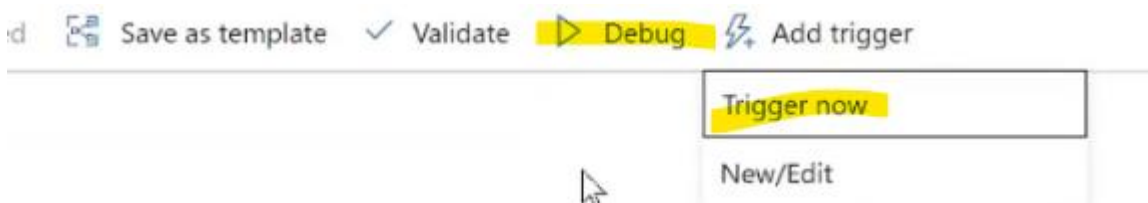
33. What is diff Linked service and Dataset in ADF?

→ LS – It provide the ability to connect with any external service outside of data factory like ADLS gen2, SQL server, Oracle DB, stg location, Databricks.

We provide URL, server name, authentication related details.

DS – it is the next step after the LS. Where we specify DB or tb name. In case of file, we provide the exact file name/folder name, etc.

34. What is the diff between debugging the PL or clicking in Trigger now option?



→ Debugging means its about debugging any PL while doing the development.

Where the PL runs end-to-end in the current branch whatever we are selected. In

Trigger Now- It will trigger Publish copy of PL.

Debug, if we are using for each loop then it will run sequentially only. In trigger now, it will multiple things in parallel.

35. In lookup activity, How do you use First row only option?

→ I check First row only when I need a single value like a watermark, and I leave it unchecked when I need a list to drive a ForEach, such as a metadata table of source tables to ingest.

Checked (single value)

- It returns one row as an object, even if the query returns more rows.
- Access it with **@activity('Lookup1').output.firstRow.ColumnName**
- Use it for a watermark or last-load date, a config value, or a row count.
- Example: `SELECT MAX(modified_date) AS wm FROM ctl.watermark` gives `@activity('Lookup1').output.firstRow.wm`

Unchecked (list of rows)

- It returns an array.
- Access it with **@activity('Lookup1').output.value** and the row count with `output.count`
- Use it when you need to loop over rows, such as a table list from a metadata table feeding a ForEach:
Items = @activity('Lookup1').output.value, then **@item().table_name** inside the loop.

- Limits: 5,000 rows and 4 MB of output. Beyond that the activity fails, so for large lists you need to split or paginate the query.

36. For each act, How do we pass element to for each and how do use it for FOR EACH loop for multiple elements?

→ Passing elements in (Items)

- Set **Items** (dynamic content) to an array.

Using each element inside the loop

- The current element is **@item()**.
- If each element is an object (a row from Lookup), use **@item().ColumnName**, for example **@item().table_name** or **@item().source_schema**.

37. What is the Fail Act, when we needed?

→ Fail act – it set status as failure then we can pass failure msg.

I use the Fail activity to stop the pipeline with a custom error message when a business or data quality check fails, such as an empty source file or a row count mismatch, so the pipeline fails clearly and alerts fire

38. What is switch act?

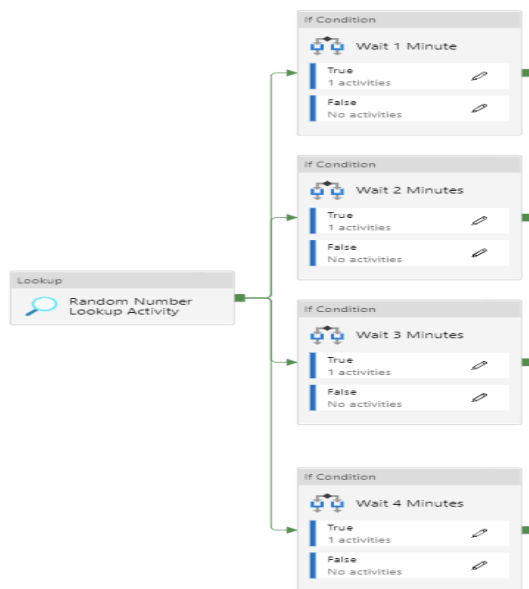
→ Its kind out of case statement.

I use Switch when a pipeline has more than two paths based on one value, like load type or source type, and I keep a Default branch with a Fail activity to catch invalid input.

We can decide source system, Default, Postgres, Csv file, etc

Switch vs If Condition

- **If Condition:** one true/false check, two paths.
- **Switch:** one value, many paths.



39. How can we pass parameter value from adf to Databricks notebook. And use it in notebook?

→

In ADF I use the **Databricks Notebook activity**. In its **Settings**, under **Base arameters**, I add name-value pairs. The value can be dynamic content, like `@pipeline().parameters.src_name` or `@item().table_name` inside a `ForEach`.

In the notebook, I read them with **widgets**: `dbutils.widgets.text` creates the widget with a default value, and `dbutils.widgets.get` reads it.

The name must match the base parameter name exactly.

To send a value back, I end the notebook with `dbutils.notebook.exit()` and read it in ADF with `@activity('Notebook1').output.runOutput`

ADF side (Base parameters)

Name	Value
src_name	<code>@pipeline().parameters.src_name</code>
load_date	<code>@formatDateTime(utcNow(),'yyyy-MM-dd')</code>

Databricks notebook side

```
dbutils.widgets.text("src_name", "")
```

```
dbutils.widgets.text("load_date", "")
```

```
src_name = dbutils.widgets.get("src_name")
```

```
load_date = dbutils.widgets.get("load_date")
```

```
# send a value back to ADF (must be a string)
```

```
dbutils.notebook.exit("SUCCESS")
```

Remember

- Parameter names are **case-sensitive** and must match on both sides.
- Values always arrive as **strings**, so cast them in the notebook, for example `int(...)`.
- `dbutils.notebook.exit()` returns only **one string**. For more values, return JSON text and parse it with `json.loads`.
- If the widget is not created in the notebook, the base parameter is ignored.

Likely follow-up: *How do you pass secrets?* "I never pass passwords as parameters. I use a Key Vault-backed secret scope and read them with `dbutils.secrets.get`."

40. How can we pass any output value or any return value from DB nb back to the pipeline and use it there?

→ We can use `dbutils.notebook.exit(str(record_cout))`

Here what ever value we pass it as JSON. That value will ooutput of the notebook. In side the PL we can use that value as o/p value of notebook.

For that we can we it

`@{activity('activity_name').output.runOutput}`

41. What is the diff Pipeline parameter or Global Parameter?

→ Both are **read-only during a run**. To change a value mid-run, use a variable.

Global para – use any PL, Define once and use every where.

	Pipeline parameter	Global parameter
Scope	Only that one pipeline	The whole data factory, any pipeline
Defined	Inside the pipeline (Parameters tab)	Manage → Global parameters
Value set by	The caller: trigger, Execute Pipeline, or debug run	Fixed at factory level, the same everywhere
Syntax	<code>@pipeline().parameters.name</code>	<code>@pipeline().globalParameters.name</code>
Typical use	Run-specific input: table name, load type, file date	Environment constants: env name, storage account, Key Vault URL

42. How can we use pipeline parameter and global parameter?

→ I use a **pipeline parameter** when the same pipeline must run with different inputs. For example, `src_table` and `load_type` are passed from the trigger or from a parent pipeline, and I read them with `@pipeline().parameters.src_table`. I use a **global parameter** for values that are the same across all pipelines but change per environment, like the environment name or storage account URL. I define it once, read it with `@pipeline().globalParameters.env`, and override it per environment in CI/CD.

Example: a path like

`@concat(pipeline().globalParameters.container, '/', pipeline().parameters.src_table)`

Gotcha: for CI/CD, tick **Include global parameters in ARM template**, otherwise they won't deploy to the other environments.

43. have you use dynamic expression in ADF? What all function you used?

→ I use pipeline parameters for run-level inputs, global parameters for environment constants, and dynamic expressions with functions like **concat()**, **formatDateTime**, **if** and **coalesce** so one pipeline is reusable across tables and environments.

"Yes, I use them daily to avoid hard-coding. Common ones:"

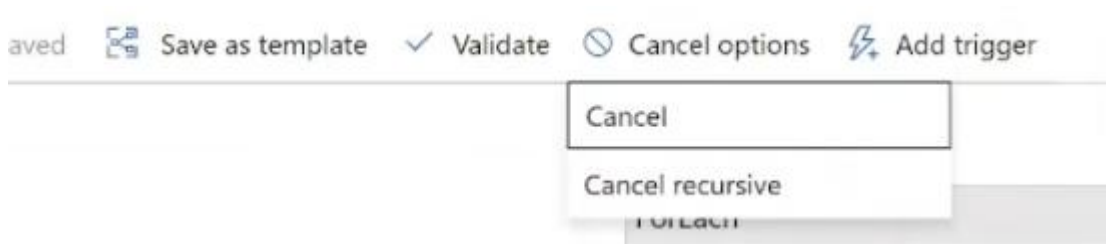
- **String:** `concat`, `substring`, `replace`, `toLowerCase`, `split`, `trim`
- **Date/time:** `utcnow()`, `formatDateTime`, `addDays`, `subtractFromTime`, `convertTimeZone`
- **Logical:** `if`, `equals`, `and`, `or`, `not`, `empty`, `coalesce`
- **Collection:** `createArray`, `length`, `first`, `last`, `contains`, `union`
- **Conversion:** `int`, `string`, `json`, `bool`
- **Math:** `add`, `sub`, `mul`, `div`, `mod`

And system references: `@activity('Lookup1').output.firstRow.col`, `@item()` in `ForEach`, `@variables('x')`, `@pipeline().RunId`, `@pipeline().TriggerTime`, `@dataset().param`.

Examples:

- Today's folder: `@concat('raw/', formatDateTime(utcnow(),'yyyy/MM/dd'))`
- Condition: `@equals(activity('Lookup1').output.firstRow.cnt, 0)`
- Default value: `@coalesce(pipeline().parameters.load_date, formatDateTime(utcnow(),'yyyy-MM-dd'))`

44. What diff cancel and cancel recursive when you are debugging something?



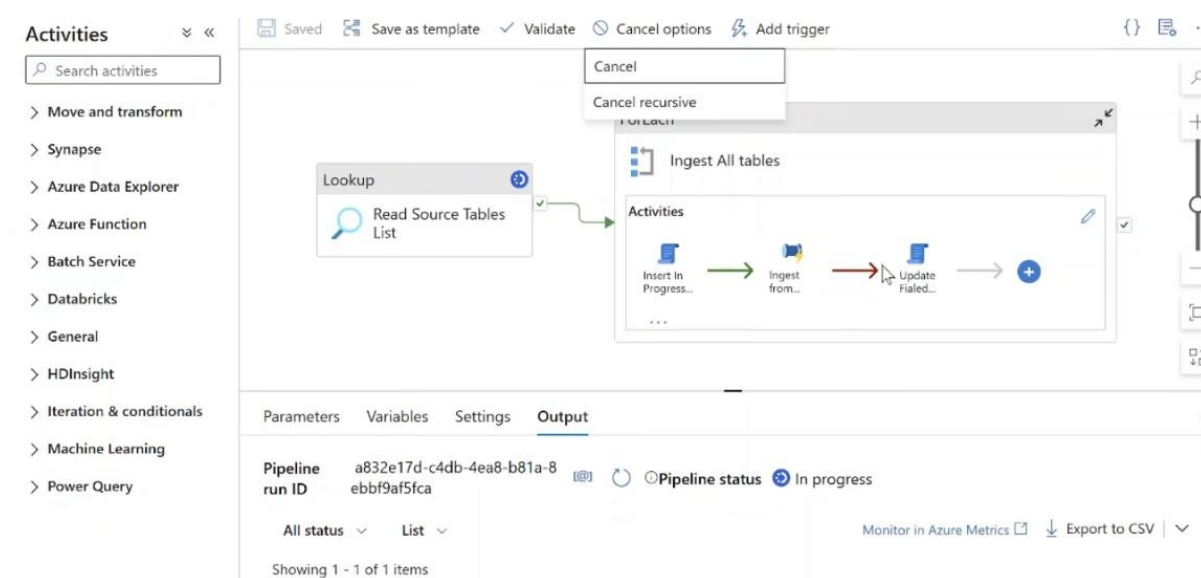
→ Cancel – It will cancel only current pipeline.

cancel recursive - It will cancel inner pipeline also with current pipeline.

45. What is pipeline runID and how can we access it dynamically within expressions?

→ It is a unique ID given to each run of PL.

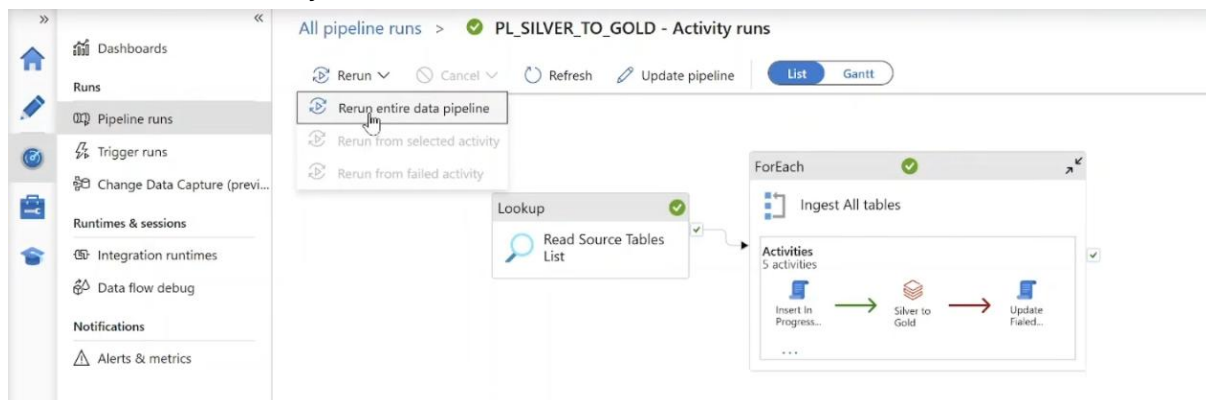
Access using --> @pipeline.runid()



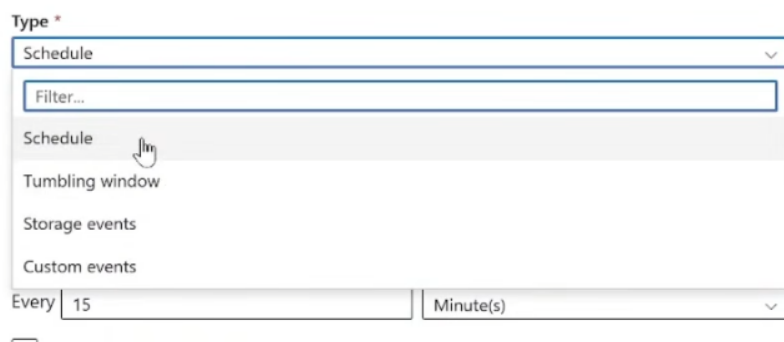
46. suppose we hv multiple activity in PL. 3 and 4th activity failed. When we have fixed the issue and if you want to run from that failed activity again in prod. When how can we do that?

→ For the, in monitor section, we get below option like

- Rerun entire data pipeline
- Rerun from selected activity
- Rerun from failed activity



47. What are the type of trigger in ADF and Databricks with use?



→ In ADF I use schedule triggers for regular loads, tumbling window for time-sliced incremental loads and backfill, and storage event triggers when a file lands. In Databricks Jobs I use scheduled or file arrival triggers, and continuous for streaming. When ADF orchestrates everything, it triggers the Databricks notebook or job, so I only schedule in one place

ADF triggers

Trigger	What it does	Use it for
Schedule	Runs on a clock (cron-like: every day at 2 AM)	Regular daily or hourly loads
Tumbling window	Fixed-size, non-overlapping, back-to-back time windows. Keeps state, supports backfill, retry and dependencies	Incremental loads by time slice, reprocessing past periods
Storage events	Fires when a blob is created or deleted in the storage account	Run when a file lands in ADLS or Blob
Custom events	Fires on an Event Grid topic event	Event-driven flows from other apps

Manual runs (**Trigger now**, API, debug) also exist, but they are not trigger types.

Key difference: schedule vs tumbling window

- **Schedule:** one trigger can start many pipelines, and it has no backfill or window state.
- **Tumbling window:** one trigger maps to one pipeline, and it passes window start and end times (`@trigger().outputs.windowStartTime`), so it's best for reliable incremental and backfill loads.
- **Databricks Jobs triggers**

Trigger	What it does	Use it for
Scheduled	Periodic or cron, with a time zone	Nightly or hourly pipelines
File arrival	Runs when new files land in a Unity Catalog volume or external location	Event-driven ingestion, often with Auto Loader
Continuous	Job runs all the time and restarts a new run when one ends or fails	Near real-time streaming jobs

Trigger	What it does	Use it for
Table update	Runs when a Unity Catalog table is updated	Downstream jobs that depend on fresh data
Manual	Run now, API or CLI	Testing, ad-hoc, external orchestrators

48. What are types of IR that we can use it in ADF?

Integration runtime setup

Network environment:

Choose the network environment of the data source / destination or external compute to which the integration runtime will connect to for data flows, data movement or dispatch activities:



Azure

Use this for running data flows, data movement, external and pipeline activities in a fully managed, serverless compute in Azure.



Self-Hosted

Use this for running activities in an on-premises / private network

[View more](#) ▾

External Resources:

You can use an existing self-hosted integration runtime that exists in another resource. This way you can reuse your existing infrastructure where self-hosted integration runtime is setup.



Linked Self-Hosted

[Learn more](#) ⓘ

→ ADF has three IR types: Azure IR for cloud sources and data flows, self-hosted IR for on-prem or private-network sources, and Azure-SSIS IR to run SSIS packages. I use Azure IR by default, and a self-hosted IR when the source is on-prem or behind a firewall.

Integration Runtime (IR) is the compute ADF uses to run activities: copying data, running data flows, and dispatching activities.

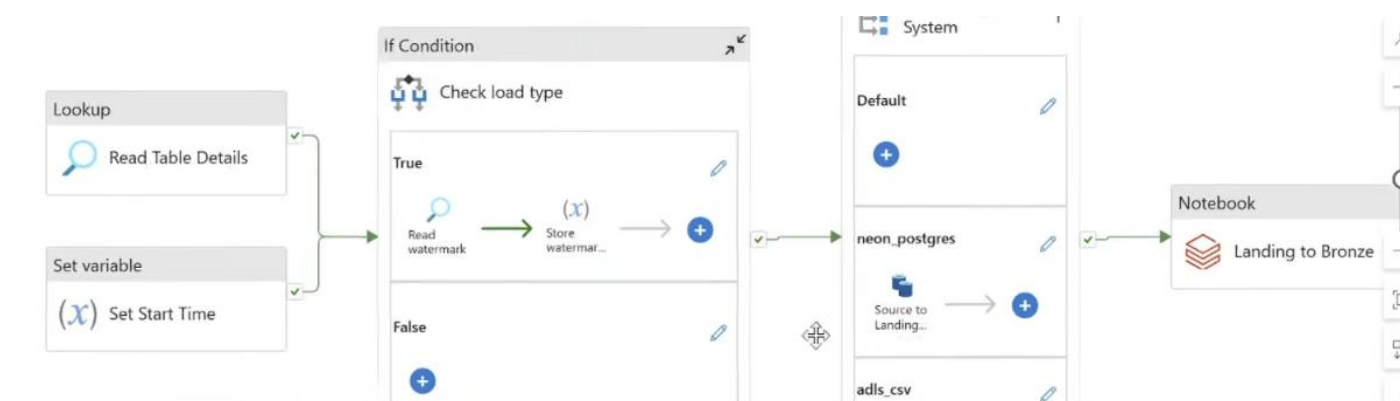
IR type	Where it runs	Use it for
Azure IR	Azure, managed by ADF	Cloud-to-cloud copy (ADLS, Azure SQL, Blob), mapping data flows, dispatching Databricks and other activities. This is the default (AutoResolveIntegrationRuntime).

IR type	Where it runs	Use it for
Self-hosted IR (SHIR)	A VM or machine you install and manage, on-premises or in a private network	On-prem or private-network sources (on-prem SQL Server, Oracle, file shares), or sources behind a firewall.
Azure-SSIS IR	A managed cluster of Azure VMs	Lifting and shifting existing SSIS packages to Azure.

49. Have you work on Meta data Framework pipeline? OR

Have ingestion multiple tables from diff source? How we are doing that?

→ We have some control tables in Separate Azure SQL Server. Based on those, we are creating reusable pipeline and notebooks. Which can ingest the data from same src or similar type of src. There we hv separate tables. Like metadata table (src_system, src_schema, src_table, src_file_form, src_patj, key_col, watermark_col). watermark table (we mention watermark value for incremental load), Audit tbl (each table we hv logid, stage_status(raw_silver, silver_gold), start_time, end_time). Using these 3 tables we have build reusable pipeline in ADF. where we read config using lookup sct. All of them are parameterize. After all loaded upto silter then we are updating the watermark column.



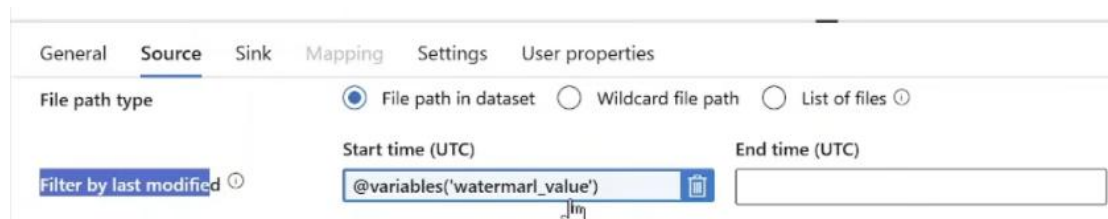
50. In incremental load, how do you make sure that every ingestion it will read incremental data only.

→ For the we are using watermark table. Which is present in azure sql server. In that we track the current ingestion timestamp. That value I am specifying that while reading the data.

51. For file base, If new file comes in the path How do you make sure only read new file from source?

→ For the we have watermark col, in that we are store util what time file has been loaded. While bring the new data that time, In copy act we have this option called **filter by last modified**.

There I specify the vault. So, whatever file has been modified after that time that only file will pick up.



52. How are u creating your notebook? So, same notebook we can use for flat file, RDNMS?

→ For Database related sources we land data into Paquet format because its very compacted.

53. What kind of audit column are mentioned for bronze, silver& gold?

→ We maintain dv_insert_ts, dv_update_ts, dv_uuid, src_name, dv_composite_key for all layer(raw_tl,atl)

54. How do you use merge statement?

→ Uses **null-safe equality** (<=>) to match rows even if id is NULL.

merge into tgt_table t

using src_table s

on t.id <=> s.id

when matched and (

not (t.name <=> s.name)

OR not (t.add <=> s.add))

Then update set (

t.name = s.name,

t.add = s.add

)

When not match

Then insert (Id, name, add) values (s.id, s.name, s.add) ;

54. If you build ur framework and you want to add new table then what are step you will follow?

→ we just have to make entry into metadata tables.

So from the same source we have to add another entry into table. That's it.

We has ls_active flag.

If you have diff src system, then create new LS, DS.

55. Have you work on any STTM documents and what kind of details are there in STTM docs?

→ Every table we have got the documents from client side or we prepared them after working with end users. Where we capture each details like source and target table name, columns name, there logic, load type.

56. Have you generated any notebook using AI?

→ yes, for gold layer we generate many notebooks using AI. 1st we prepare the STTM document then we provide that doc to **Genie Code** and ask him to create new notebook for this.

Initial version he will create then we review the notebook, refine the logic again .

57. How are you using framework to run multiple notebooks in gold layer?

→ We have the same metadata tables that we can use for both silver and gold layer.

For gold layer also we specify the table names and other details into our metadata DB. Then we are using lookup activity. Iterate over it in for each. Then call the notebook using **Notebook activity dynamically**.

58. Are you doing full load or incremental load for your gold tables?

→ for small tables we are doing full load.

Big tables, we have driving the table. Form that table, we read the data based on incremental logic which we put in watermark. We read the incremental data. That is the driving table and big gold table.

59. What kind of options are you using in your pipeline to handle failures?

→ We are using varous options like

For each and every activity, we are configuring the retry options that if there is some temporary or quick failure that so it can retry the same activity again.

And from overall design perspective we have kept all the data lrealted operations into a separate pipeline. So, even if the single activity fails in a pipeline. In outter pipeline we are able to capture it and log it into properly.

And in turms of auditing and logging, we are logging the information in very detail including the ADF run ID (to trece back the things), start time, end time, everything we are logging.

60. What kind of options we get at the activity level to handle failures?

→ We have 3 options

- 1.. timeout – for very long running we can specify the timeout.
- 2.. Retry – How may times we want to retry
- 3.. Retry interval – How much gap we want to provide between different runs

General Settings User properties

Name * [Learn more](#)

Description

Activity state ☒ Activated ☐ Deactivated

Timeout

Retry

Retry interval (sec)

61. How you find error message or what is wrong with the pipeline?

→ In our notification we get the runID. Using runID we can go to **MONITOR** Section of databricks, find the particular runID and then in the activity output we can check out the error msg.

62. After fixing your problem, do we entirely run job or do you rerun from the failed activity?

→ It depends on scenario,

If some ADF activity related failure is there for example copy data or something then we apply fix and need to **publish** the fix, save it. And again, run from the start.

If failure comes from the notebook, then we can fix the issue inside the notebook and we can rerun from the failed activity as well.

It also depends on the, How much time it will take to run from the start or fail.

What impact is there like if it can create duplicate data by rerunning entire pipeline then we will avoid it and run from its failed.

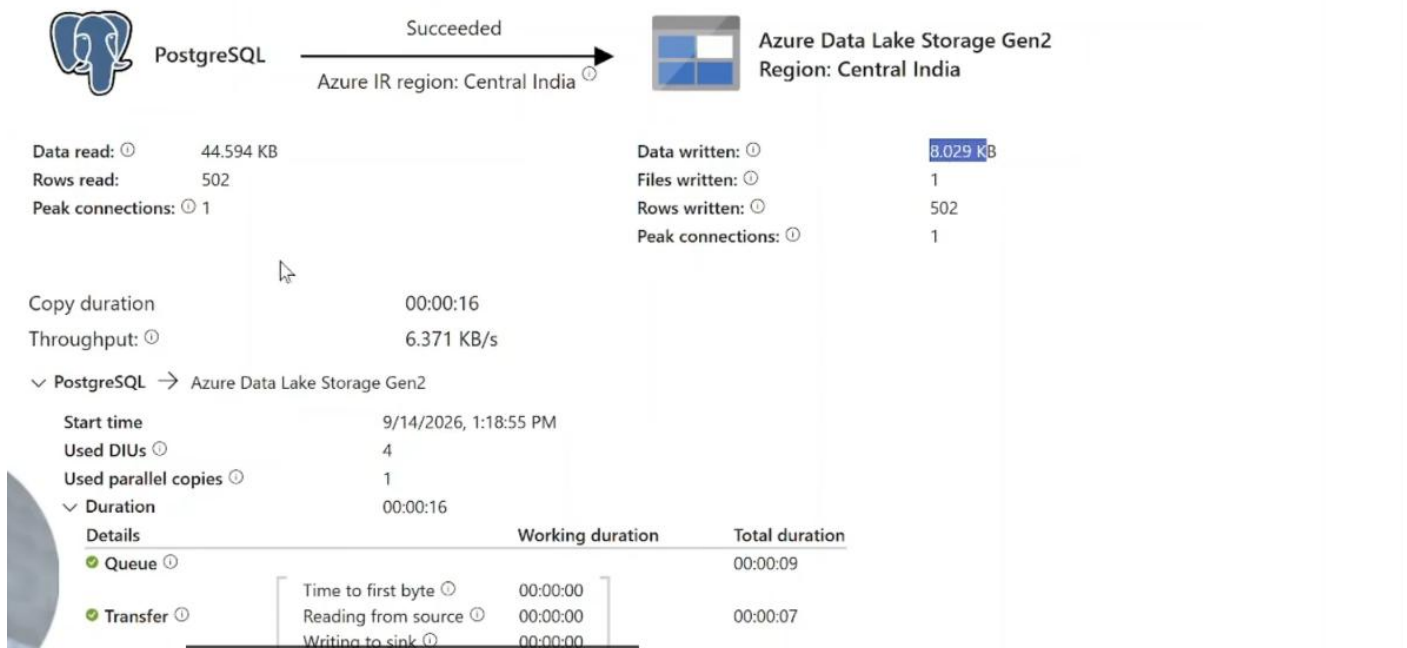
63. Have you monitor the copy data activity and there what kind of details we can figure out from the monitoring section?

→ For copy data activity, we get specific details section in the monitoring section. There we can monitor how many rows has been read and how much data volume has been read. How many parallel connection they used, what is speed of data, duration of speed. All of this we will see in output of the copy data activity.

Details  Refresh

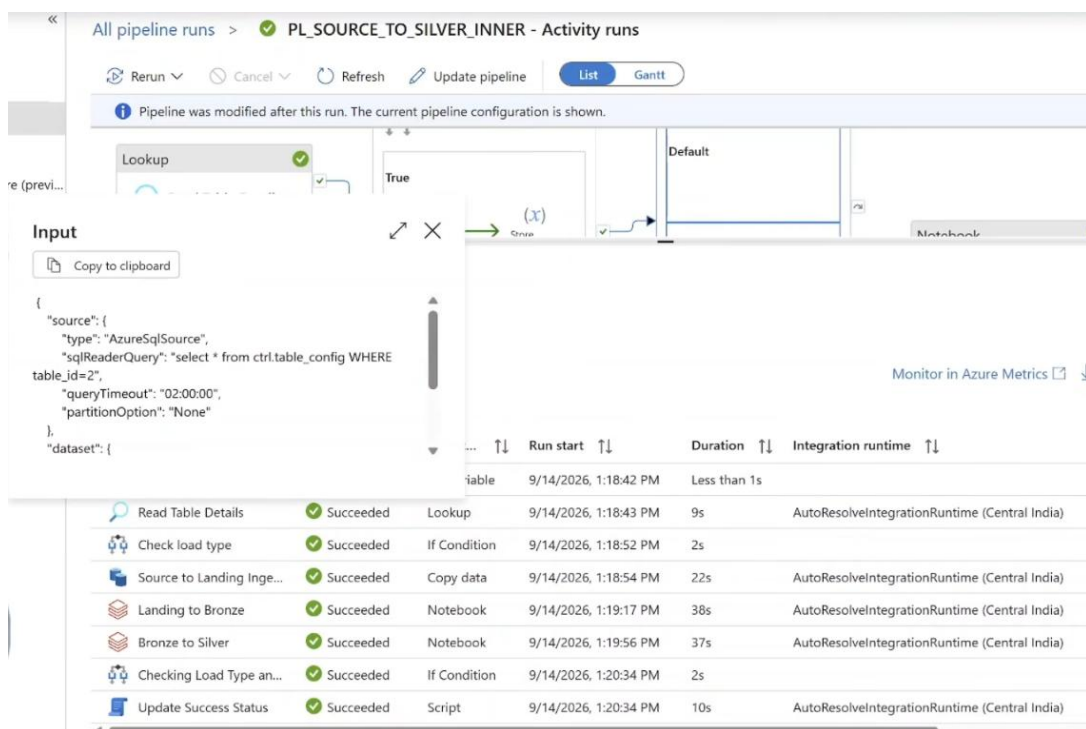
[Learn more on copy performance details from here.](#)

Activity run id: 155320ac-44a7-4916-8385-e022ab16f4a8



64. How can we know that at runtime what exact query was used?

→ In monitor section, we can checkout the input of that particular activity. There we can check the exact query that has been prepared using dynamic expression.



65. Which service are you using for email notification?

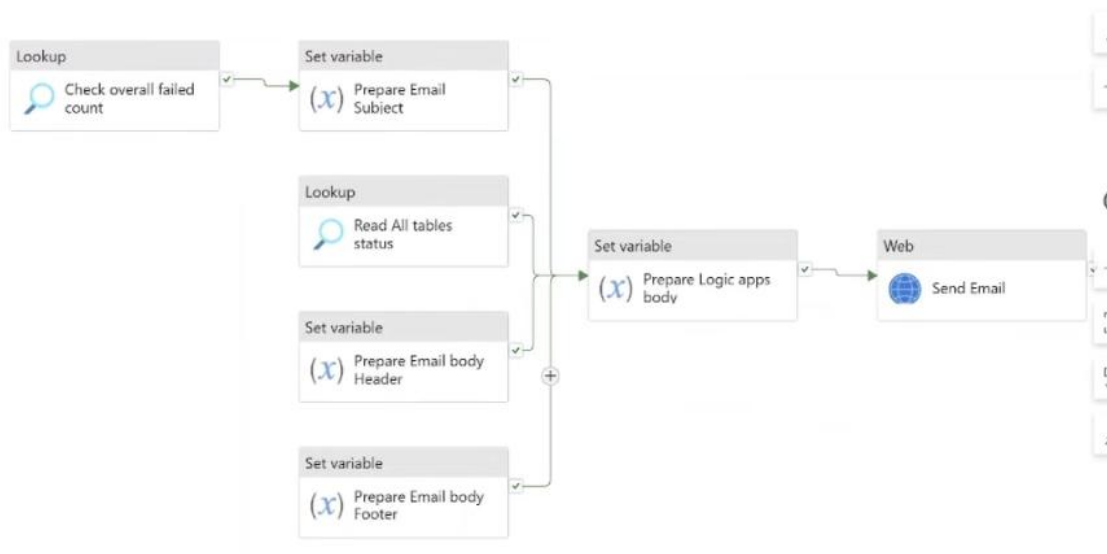
→ We have built our own logic apps workflow and that we are calling it in ADF using web Activity.

66. If you want to send consolidated email(bronze, silver, gold mail in one mail), how you will send?

→ For that, we have audit table, where in we insert status and records count for every table.

67. Can you explain design of that email notification pipeline? Or how you created reusable pipeline for email notification?

→ Our pipeline only take RunID, based on that it fetches all information about audit table and also t check overall status whether there is a single at least one failure or everything is successful. Based on that we prepar email subject, header, footer and separate status table, a separate a set variable in separate set variable activities anf then we prepare consolidated body that we can send to logic apps.



Followup – What kind of parameterization you have done for the logic apps?

→ We have parameterized the subject, email recipient and body part. That we are sending it in three different parameters.

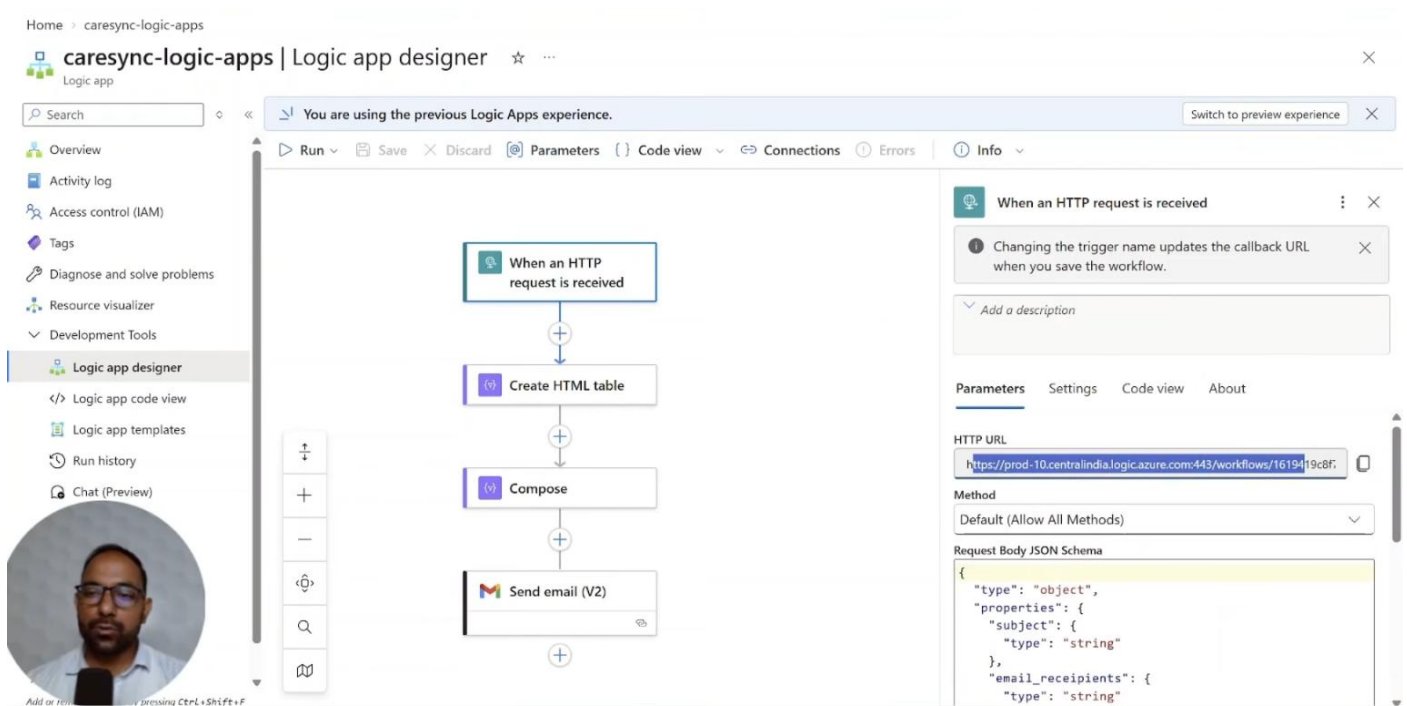
Pipeline expression builder

Add dynamic content below using any combination of expressions, functions and system variables.

```
{
  "subject": "@{variables('email_subject')}",
  "email_recipients": "@{pipeline().globalParameters.
    email_recipient}",
  "body_header": "@{variables('body_header')}",
  "body_footer": "@{variables('body_footer')}",
  "status_table": "@{activity('Read All tables status').output.
    value}"
}
```

68. What you work o logic app side and there what kind of activities you are using?

→ Yes, I have worked. There mainly like we are using for activities.



69. How you are storing credentials securely in azure key vault?

→ we configure the azure key vault and whatever passwords or creds we have to store . we store in key vault secrets and created a linked service for KV for our ADF. Read them through Azure Key Vault section in inked service by selecting the linked service and selected name. We can use directly without showing actual paasword.

70. What kind of authentication you are using to connect ADLS Gen2 and How can we setup connection with ADLS with ADF?

→ We are using system assigned managed identity (stg -> IAM -> role assignments -> **stg blob data contributor**[Managed Identity]). We don't need to provide any creds from connecting with Linked service.

71. If you have new storage account, How can you connect that storage account in Databricks and what are the step to follow to read/write data?

→ For that We are using **Unity-catalog-access-connector** azure service. Using that

1st we have to create credential In Catalog of Databricks workspace using same managed identity of that connector.

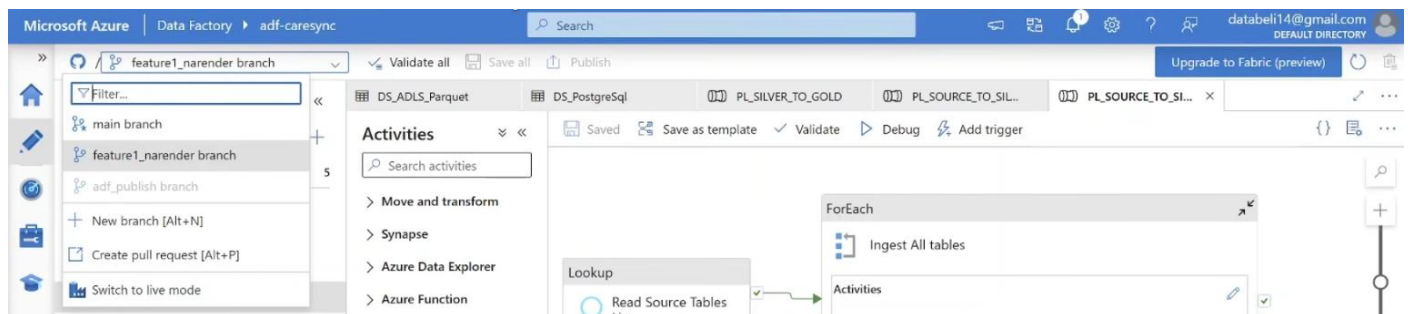
2nd For that managed identity, we have given **stg blob data contributor** access on the ADLS Side.

3rd Using that creds, we will create **External Location** in Unity catalog. Which uses those creds and connect to our ADLS account

4th For read/write, we created **Volume** using external location. In that volume, we can directly see the file of that new ADLS container

72. How do you follow CI/CD process on ADF side?

→ Whenever we want to work on new feature. We create new branch from main branch called feature branch. In feature branch, we did the changes, apply those changes, test them. Then we create PR to take those change to the QA branch. PR is review by senior's. after review PR is merge to QA. then the change are publish into final branch.

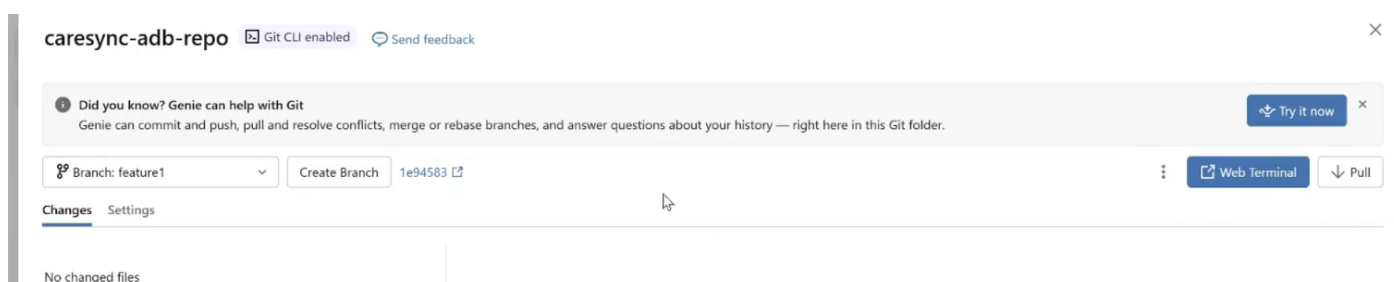
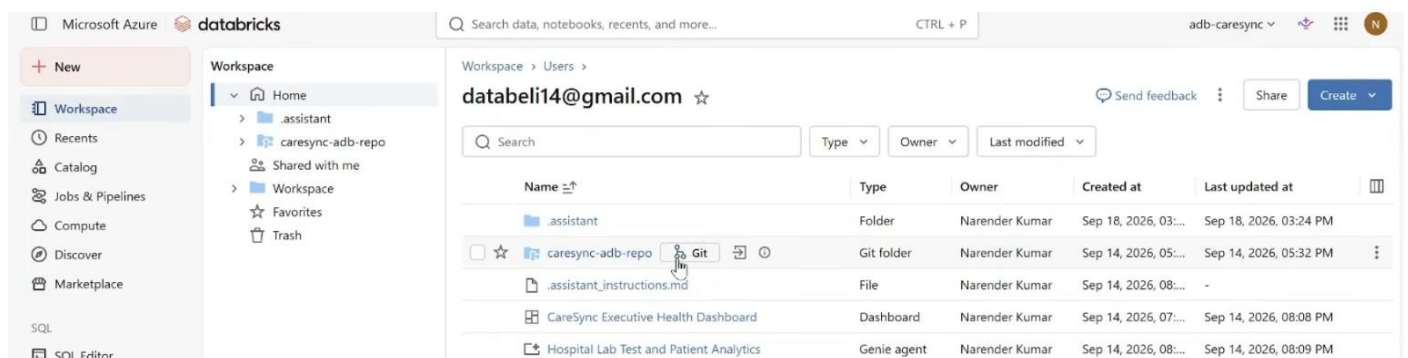


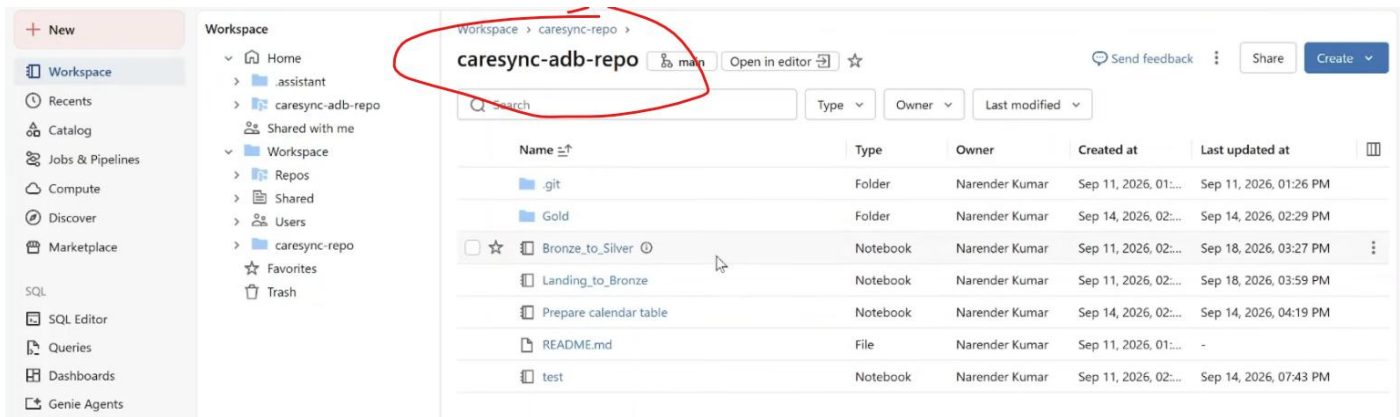
73. How do you follow CI/CD process on Azure Databricks side?

→ Our final Notebooks are present in main workspace folder. Which is outside of the all user related folders. In that folder, we have connected it through git and created a git folder. Whenever we want to work in new feature. Our individual users folder we clone the git folder. There we create new branch out of main branch called Feature branch. In that, we work on all changes. Then we push our change and create PR from dev to QA. PR is review by seniors. after review PR is merge to QA. then the change are publish into final branch.

After change done from git side or DevOps side then in final location again we will goto git folder. There we pull our latest changes which are available on git side. And those change are available for Pipeline.

This process we follow for CI/CD process from development perspective.

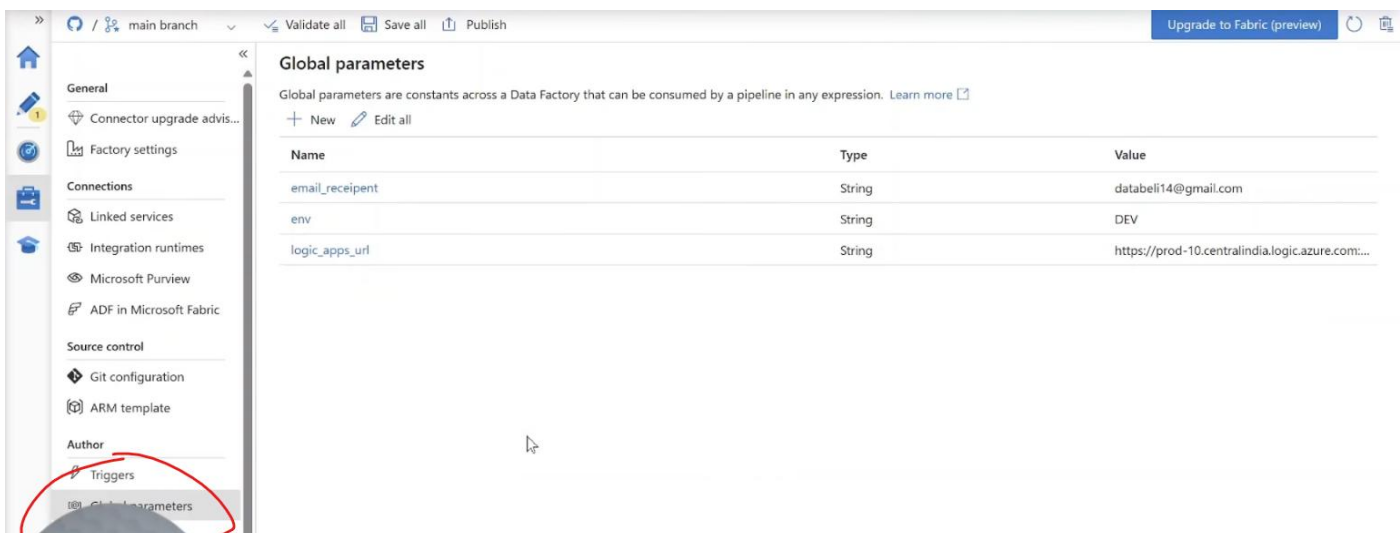




74. How you changes deployed from 1 env to another env? What kind of things from your side you take care?

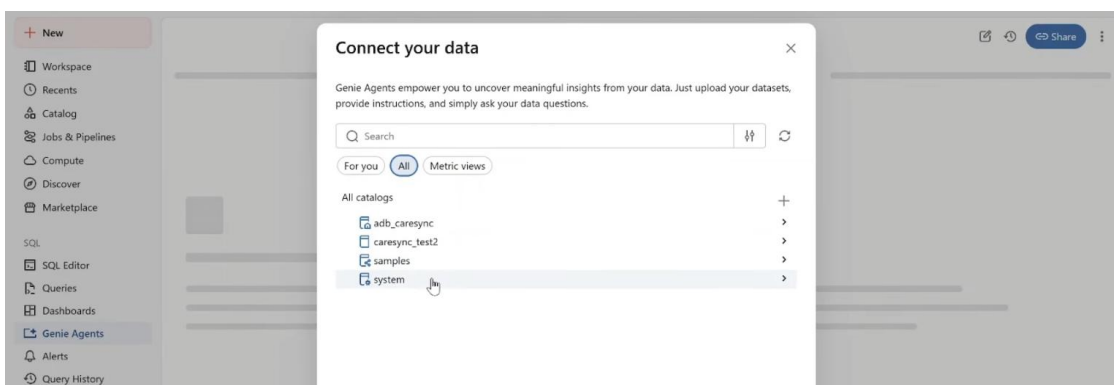
→ For CI/CD process we have DevOps Engineer in the team. They have built the deployment pipeline which can take changes from one workspace and then deploy them to another workspace.

From our side, we are creating out pipeline and notebooks. So that those can be used in any of the env without any changes. We are using global parameters.



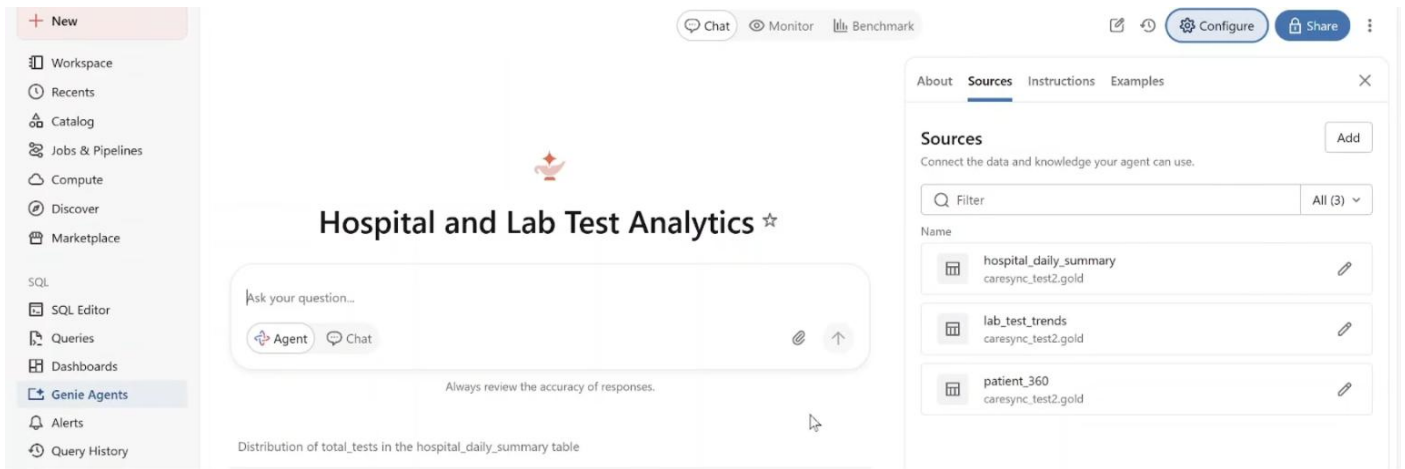
75. How do you create Genie agent?

→ Goto Genie agent section and we click on new. Then we can select the table that we want to connect or metric views whatever artifacts we want to build on it then we click on connect. Initial version he will create. Agent has capability to give any answer based on nature lang question. Ready to use.



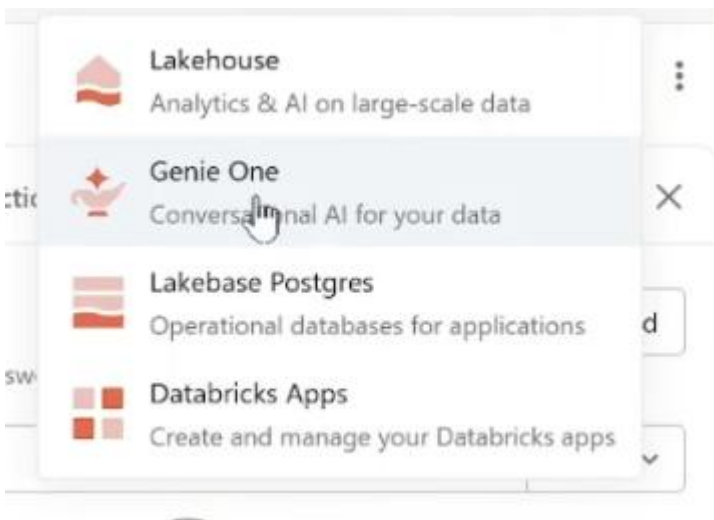
76. what is Genie agent and How it is used?

→ In Genie agent, we give some info like tables, views. All those things provide in **sources**. Then it has capability to give any answer based on natural language question. We can customize the response.

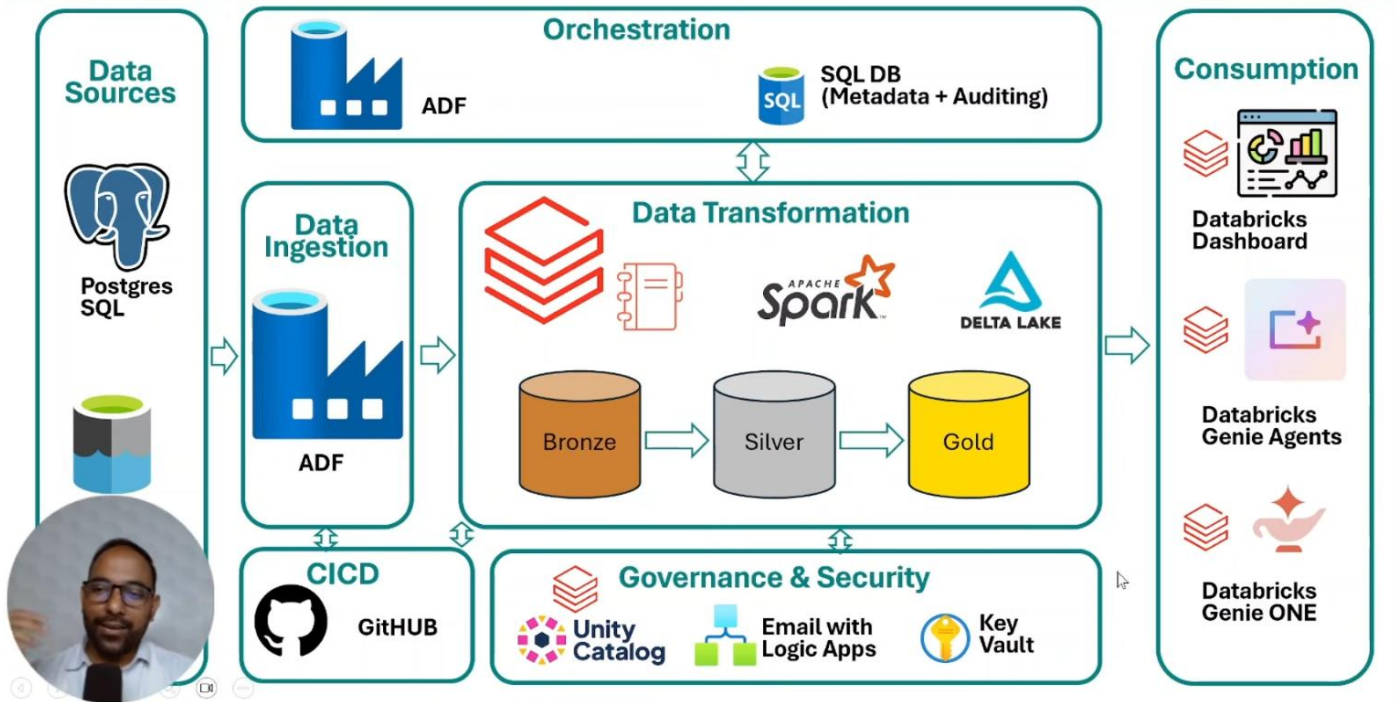


77. What is Databricks Genie One ?

→ it is dedicated separate UI from Databricks side. Which is specially design for business user. They don't need to run any query or see any technical info.



CareSync - Architecture



Data Engineer Interview Q&A — Mayuresh Patil

Format: each question has a **Short Answer** (30-second version) and a **Detailed Answer** (interview script, first person). Tip: replace the numbers with your exact figures if they differ. Never quote a metric you can't explain.

Q1. How do you decide the partition granularity to balance query performance and data management overhead?

Short Answer I partition on the column that is most frequently used in filters and has low-to-medium cardinality, usually a date. I aim for partitions of roughly 128 MB to 1 GB of data. Too many small partitions create small-file and metadata overhead, and too few lose pruning benefits.

Detailed Answer "I start from the query patterns. If most reports filter by date, I partition by date. Then I check data volume per partition. If daily data is only a few MB, daily partitioning creates thousands of tiny files, so I move to monthly or use a coarser grain. If a day holds hundreds of GB, I add a second level or use hidden partitioning.

My rules of thumb:

- Partition column should have low cardinality. Never something like customer_id.
- Each partition should be around 128 MB to 1 GB at minimum.
- Partitions that are too small cause small-file problems, slow listing and heavy metadata.
- For high-cardinality filter columns, I don't partition. I use Z-Ordering or clustering instead.

In Iceberg I used hidden partitioning like `days(event_ts)`, so I could change granularity later without rewriting queries. In Delta, I partition by date for large fact tables and use Z-Order on columns like account or business unit. I validate with the Spark UI by checking files scanned and data skipped before and after."

Delta: partition by date, then check partition sizes

```
(df.write.format("delta")
```

```
.partitionBy("load_date")
```

```
.mode("overwrite")
```

```
.save(path))
```

```
spark.sql("DESCRIBE DETAIL delta.`{}`".format(path)).select("numFiles", "sizeInBytes").show()
```

```
-- Iceberg hidden partitioning
```

```
CREATE TABLE sales (id BIGINT, event_ts TIMESTAMP, amount DOUBLE)
```

```
USING iceberg
```

```
PARTITIONED BY (days(event_ts));
```

Q2. Tell me about your experience with CI/CD pipelines, specifically using GitHub for data engineering projects.

Short Answer I use Git branching with pull requests for code review, and GitHub-based CI to run linting and unit tests on PySpark code. Merges to main trigger deployment of notebooks, jobs and ADF artifacts to higher environments, with secrets kept in Key Vault.

Detailed Answer "In my projects, all code lives in GitHub: PySpark modules, Databricks notebooks, ADF configs and SQL scripts. We follow a feature branch workflow. A developer raises a pull request into develop. On every PR, CI runs flake8 or black for linting and pytest unit tests on transformation functions using a local Spark session. Another engineer must approve before merge.

After merge to develop, the pipeline deploys to the Dev/QA workspace. Promotion to Prod happens from main with an approval gate. Environment-specific values such as storage paths, cluster configs and connection details are parameterized, and credentials come from Azure Key Vault, never from the repo.

The benefits were consistent deployments, fewer manual errors, and a clear audit trail, which matters in regulated environments."

```
# .github/workflows/ci.yml
```

```
name: ci
```

```
on:
```

```
  pull_request:
```

```
    branches: [develop, main]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-python@v5
```

```
        with: { python-version: "3.10" }
```

```
      - run: pip install -r requirements.txt pytest flake8
```

```
      - run: flake8 src/
```

```
      - run: pytest tests/ -q
```

Q3. How do you automate deployment of data pipelines and ensure continuous delivery from development to production?

Short Answer I keep everything as code, parameterize per environment, and use a CI/CD pipeline that tests, packages and deploys to Dev, then QA, then Prod with approvals. Secrets come from Key Vault, and I add post-deploy smoke checks and a rollback path.

Detailed Answer "My approach has four parts.

1. **Everything as code.** Notebooks, PySpark packages, ADF pipelines, job definitions, and config files all sit in Git.
2. **Environment parameterization.** One codebase, with separate config for Dev, QA and Prod. In ADF I use global parameters and linked services per environment. In Databricks I use job parameters and config files.
3. **Pipeline stages.** Lint and unit tests, then deploy to Dev, run an integration test on a sample dataset, deploy to QA, then manual approval for Prod.
4. **Safety.** Secrets come from Key Vault through secret scopes. After deployment I run a smoke test, such as a row-count or schema check. For rollback, I redeploy the previous tagged release since every release is versioned.

For ADF, I publish ARM templates from the collaboration branch and deploy them with parameter overrides per environment. For Databricks, I deploy jobs and notebooks with the Databricks CLI or Asset Bundles."

Example deployment step

```
databricks bundle validate -t prod
```

```
databricks bundle deploy -t prod
```

Q4. What considerations do you take into account when deciding whether to broadcast a join, especially with varying data sizes?

Short Answer I broadcast when one side is small enough to fit comfortably in executor and driver memory, typically under a few hundred MB, and the other side is large. It avoids shuffling the big table. I check actual size after filters and watch for skew and memory limits.

Detailed Answer "A broadcast join sends the small table to every executor, so the large table doesn't need to be shuffled. That's a big saving. Before broadcasting I consider:

- **Size after filters and column pruning**, not the raw table size.
- **Memory available** on the driver and executors, since the table is collected on the driver first.
- **Join type.** The broadcast side has restrictions depending on join type, e.g. you can't broadcast the preserved side of an outer join.
- **Stability of size.** If the table can grow, a hard-coded broadcast can break later.

By default Spark auto-broadcasts under `spark.sql.autoBroadcastJoinThreshold` (10 MB). With AQE enabled, Spark can also convert a sort-merge join to a broadcast join at runtime based on actual shuffle statistics. For clearly small lookup tables I use an explicit hint."

```
from pyspark.sql.functions import broadcast

result = fact_df.join(broadcast(lookup_df), "product_id", "left")
```

Q5. How do you define a "small" lookup table, and what are the drawbacks if you broadcast a table that isn't truly small?

Short Answer Small means it fits comfortably in driver and executor memory, usually tens to a couple hundred MB in memory, not on disk. If it's larger, you risk driver OOM, executor memory pressure, slow broadcast, and failed jobs.

Detailed Answer "I define 'small' by in-memory size, not file size. Parquet is compressed, so a 50 MB file can expand to several hundred MB in memory. My working rule is: a lookup is small if its deserialized size is well under the driver and executor memory available, typically under about 100 to 200 MB in my projects, though the default threshold is only 10 MB.

If you broadcast a table that isn't small:

- The driver collects the whole table first, which can cause **driver OOM**.
- Every executor holds a full copy, which increases **executor memory pressure** and GC.
- Broadcast **timeouts** can occur since the data must be shipped to all nodes.
- Instead of a speedup, the job becomes slower or fails.

So I measure first, filter and select only needed columns, and if it's borderline, I let Spark choose or I use a sort-merge join."

Q6. The driver tries to load a non-small table and hits OOM. How do you mitigate this and decide the broadcast threshold when data sizes are dynamic?

Short Answer Don't hard-code broadcast hints for tables that can grow. Rely on AQE and a sensible threshold, reduce the table before broadcasting, and fall back to sort-merge join when size exceeds a limit. Also size driver memory properly.

Detailed Answer "To mitigate the risk:

1. **Remove forced hints on tables with variable size.** Let Spark decide using `autoBroadcastJoinThreshold` and AQE, which uses real runtime statistics.
2. **Shrink the table first.** Apply filters, select only join and needed columns, and deduplicate.
3. **Set limits.** Keep `spark.sql.autoBroadcastJoinThreshold` modest, increase `spark.driver.memory` and `spark.driver.maxResultSize` if needed, and tune `spark.sql.broadcastTimeout`.
4. **Add a size guard.** For dynamic tables I check size at runtime and broadcast only if below my limit, otherwise use a normal join.
5. **Monitor.** I watch the Spark UI for broadcast size and driver memory, and set alerts on failures.

To pick the threshold, I test with representative data, measure in-memory size, and keep it at a fraction, say less than a third, of driver and executor memory."

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024) # 50 MB
```

```
# Size guard for dynamic tables
```

```
size_bytes = lookup_df._jdf.queryExecution().optimizedPlan().stats().sizeInBytes()
```

```
if size_bytes < 100 * 1024 * 1024:
```

```
    result = fact_df.join(broadcast(lookup_df), "key")
```

```
else:
```

```
    result = fact_df.join(lookup_df, "key")
```

Q7. How did you achieve the 40% reduction in query latency?

Short Answer By optimizing the physical layout of the Delta tables: Z-Ordering on commonly filtered columns, file compaction to fix small files, and better clustering and partitioning. That improved data skipping and cut the data scanned by reports.

Detailed Answer "In FinOps Dataverse, 300+ business reports ran on Delta tables. I first profiled slow queries in the Spark UI and found two problems: too many small files from incremental loads, and queries scanning far more data than needed.

I fixed it in steps:

1. **File compaction** using OPTIMIZE to merge small files into larger ones.
2. **Z-Ordering** on the columns most used in report filters and joins, which improved data skipping.
3. **Right partitioning** so common date filters pruned partitions.
4. **Maintenance** with scheduled OPTIMIZE and VACUUM so performance didn't degrade over time.

I measured before and after using the same report queries. Latency dropped by about 40%, and files scanned dropped significantly."

```
OPTIMIZE finance.fact_ledger
```

```
ZORDER BY (company_code, posting_date);
```

```
VACUUM finance.fact_ledger RETAIN 168 HOURS;
```

Q8. What other techniques, like Z-ordering or clustering, did you use to optimize Delta Lake workloads?

Short Answer Besides Z-Ordering and clustering, I used OPTIMIZE compaction, partition pruning, data skipping through statistics, auto-optimize settings, caching where useful, and VACUUM for cleanup. I also tuned Spark for joins and shuffles.

Detailed Answer "Apart from Z-Order and clustering, I used:

- **OPTIMIZE and auto compaction** to control small files, with optimized writes enabled on frequently written tables.
- **Partition pruning** through sensible partition columns.

- **Data skipping stats.** Delta collects min and max for the first columns, so I keep filter columns early in the schema when appropriate.
- **VACUUM** to remove old files and control storage.
- **Efficient MERGE.** I added partition filters in the merge condition to limit files touched.
- **Spark tuning.** Broadcast joins for small dimensions, AQE for skew and shuffle partitions, and avoiding unnecessary wide transformations.
- **Selective columns** and pushing filters early.

Z-Order helps with multi-column filters, while liquid clustering is more flexible when query patterns change, since clustering keys can be changed without rewriting the whole table."

```
ALTER TABLE finance.fact_ledger SET TBLPROPERTIES (
```

```
'delta.autoOptimize.optimizeWrite' = 'true',
```

```
'delta.autoOptimize.autoCompact' = 'true'
```

```
);
```

```
MERGE INTO tgt t USING src s
```

```
ON t.id = s.id AND t.load_date = s.load_date -- partition filter limits scan
```

```
WHEN MATCHED THEN UPDATE SET *
```

```
WHEN NOT MATCHED THEN INSERT *;
```

Q9. Tell me about a challenging data migration you worked on and how you approached it.

Short Answer I migrated 20+ TB of pharmaceutical data to Apache Iceberg on Amazon S3. I handled it in phases with schema mapping, parallel validation, incremental loads and a controlled cutover, so there was no data loss and little downtime.

Detailed Answer "The most challenging one was the e-Clinical Data Platform migration, modernizing 20+ TB of pharmaceutical data on S3 using PySpark and Apache Iceberg.

Challenges: large volume, multiple source systems, strict data accuracy needs, and minimal downtime.

Approach:

1. **Assessment.** Profiled sources, mapped schemas and identified dependencies.
2. **Design.** Chose Iceberg for ACID, schema evolution and hidden partitioning.
3. **Historical load.** Migrated in batches by table and date range, with parallelism tuned so we didn't overload sources.
4. **Incremental sync.** Used incremental loads to capture changes during migration.
5. **Validation.** Row counts, checksums and sample comparisons between source and target.
6. **Cutover.** Ran old and new in parallel, then switched consumers after sign-off.

The result was a successful migration with verified data, and a platform that supported schema changes and faster processing afterward."

Q10. How do you ensure data integrity across sources like Oracle, Snowflake and REST APIs, especially with millions of records daily?

Short Answer I use schema validation, reconciliation checks such as row counts and checksums, idempotent loads, deduplication on keys, and audit logging with alerts. For APIs I add pagination, retries and incremental watermarks.

Detailed Answer "With 10M+ records daily from Oracle, Snowflake, REST APIs and Smartsheet, I built integrity checks into every stage:

- **Schema validation** at ingestion, with rejected records routed to a quarantine location.
- **Reconciliation.** Compare source and target row counts, and checksums or aggregates on key columns.
- **Idempotent loads.** Using MERGE or overwrite-by-partition so a rerun doesn't create duplicates.
- **Deduplication** using business keys and ordering by timestamp.
- **Incremental watermarks** so we neither miss nor reload records.
- **REST APIs.** Pagination, retries with backoff, and checking total counts when available.
- **Audit tables** logging run id, counts, status and duration, with alerts on failures or mismatches.

This is how we kept pipeline availability at about 99.9%."

```
src_count = src_df.count()

tgt_count = spark.table("silver.orders").where(f"load_date = '{run_date}'").count()

if src_count != tgt_count:

    raise Exception(f"Count mismatch: source={src_count}, target={tgt_count}")
```

Q11. What challenges did you face implementing ETL pipelines in regulated environments like pharma, and how did you overcome them?

Short Answer The main challenges were data accuracy, traceability, access control and change management. I addressed them with validation checks, lineage and audit logs, role-based access with secrets management, and controlled, reviewed deployments.

Detailed Answer "In pharma, data must be accurate, traceable and secure. The challenges and my solutions:

1. **Data accuracy.** Added validation and reconciliation at each layer so errors were caught early.
2. **Traceability.** Maintained audit logs and lineage, so we could show where a record came from and how it changed. Iceberg snapshots and time travel also helped.
3. **Access control.** Role-based access, least privilege and no hard-coded credentials.
4. **Change control.** All changes went through Git, code review and approvals before production.
5. **Master data consistency.** Golden Record and XREF logic resolved the same entity across systems.
6. **Reliability.** Retries, monitoring and alerting to keep availability high.

The result was reliable pipelines that stood up to audits and gave downstream teams trusted data."

Q12. How does the Python automation for SQL configuration generation work, and what were the benefits?

Short Answer I built a Python generator that reads a metadata or config file for a new source and produces the SQL and pipeline configuration automatically from templates. It removed repetitive manual work and cut onboarding time for new pipelines.

Detailed Answer "Earlier, onboarding a new table meant manually writing SQL for extraction and transformation plus config entries, which was repetitive and error-prone.

I automated it:

1. **Input.** A simple metadata file (CSV/JSON/Excel) listing source table, columns, keys, load type and target.
2. **Templates.** Jinja2 templates for SQL and config structures.
3. **Generation.** A Python script reads metadata, validates it (for example with Pydantic), and renders the SQL and config files.
4. **Output.** Ready-to-use files that plug into the ingestion framework.

Benefits: much less manual effort, consistent standards, fewer human errors, and faster onboarding. Adding a new table became a metadata change rather than new code."

```
from jinja2 import Template
```

```
tpl = Template("""
SELECT {{ cols | join(', ') }}
FROM {{ source_table }}

{% if load_type == 'incremental' %}WHERE {{ watermark }} > '{{ last_run }}'{% endif %}
""")
```

```
sql = tpl.render(cols=["id", "name", "updated_at"],
                 source_table="SALES.ORDERS",
                 load_type="incremental",
                 watermark="updated_at",
                 last_run="2026-01-01")
```

Q13. How do you manage migration of SAP financial data to both Iceberg and Snowflake, ensuring consistency?

Short Answer I read the data once, apply the same transformation logic, and write to both targets from that single dataframe in the same run. Then I reconcile counts and aggregates between the two targets, and drive both from one config and run id.

Detailed Answer "The key principle is a **single source of truth for transformation**. I extract the SAP data once, apply the transformations once, and then write the same dataframe to both targets. That avoids logic drifting between two separate pipelines.

Steps:

1. **Read and transform once**, cached or persisted so both writes use identical data.
2. **Write to Iceberg** (S3) and **Snowflake** using the same run id and load timestamp.
3. **Same schema mapping and data types**, defined in one config, with care for type differences such as decimals and timestamps.
4. **Reconcile** after load: row counts, sums of key financial columns, and min/max dates on both targets.
5. **Idempotency**. Use merge or partition overwrite so reruns are safe.
6. **Audit** results in a log table, with alerts on mismatch."

```
df = transform(read_sap()).persist()
```

```
df.writeTo("iceberg_catalog.finance.gl_entries").using("iceberg").append()
```

```
(df.write.format("snowflake")
```

```
.options(**sf_options)
```

```
.option("dbtable", "GL_ENTRIES")
```

```
.mode("append").save())
```

Q14. How do you handle discrepancies or validation challenges when migrating to two targets at once?

Short Answer I reconcile both targets against the source and against each other using counts, aggregates and checksums. If a mismatch appears, I identify which target diverged, fix and reload that target only, and rely on idempotent writes so reruns are safe.

Detailed Answer "Writing to two targets means one can succeed while the other fails, so I plan for partial failure.

Prevention

- Same dataframe and same run id for both writes.
- Explicit type mapping, because Iceberg and Snowflake treat decimals, timestamps and nulls differently.

Validation

- Compare source vs Iceberg vs Snowflake: row counts, sums on amount columns, distinct key counts, and min/max dates.

- Row-level hash comparison for sampled or high-risk tables.

Handling discrepancies

1. Check the audit log to see which target failed or diverged.
2. Find the cause: type conversion, rounding, nulls, duplicates, or a partial write.
3. Reload only the affected target for the affected partition, using merge or partition overwrite so there are no duplicates.
4. Re-run validation and record the outcome.

I also set alerts so mismatches are caught the same day, not weeks later."

-- Compare the two targets

```
SELECT 'iceberg' AS tgt, COUNT(*) AS cnt, SUM(amount) AS total FROM iceberg.finance.gl_entries WHERE  
load_id = :run_id
```

```
UNION ALL
```

```
SELECT 'snowflake', COUNT(*), SUM(amount) FROM GL_ENTRIES WHERE LOAD_ID = :run_id;
```

Quick Reminders Before the Interview

- Lead with the **problem**, then **action**, then **result with a number**.
- Be ready to explain *why* for each choice (why Iceberg, why Z-Order, why broadcast).
- Your resume says 3.5+ years, so say "3.5+ years" or "close to 4 years" consistently.
- If you are asked about something you did not personally build, say "my team did X, my part was Y."